Universidad Mariano Gálvez de Guatemala Sede de Amatitlán

Curso de Programación II

Délmar Leonel Martínez Oliveros

31 de Octubre de 2026

PideYa — Documentación Técnica

(Proyecto Final)

Integrantes del Proyecto	
Carnet	Nombre del Alumno
6590-25-3736	Dilan Jeanncarlo Carrera Pineda
6590-25-20916	Mario José Morales Pineda
6590-25-17053	Alex David Tahuite Duarte
6590-25-22811	José André Pérez Morales
6590-25-20019	Jesse Derek Rudeen Crispin

Tabla de Contenido

Resumen 5
Hallazgos principales 5
Contenido 5
Etiquetas de certeza 5
Índice de documentación 5
Capítulo 1. Presentación del proyecto 6
1.1 Nombre y descripción 6
1.2 Problema que busca resolver 6
1.3 Objetivos 6
1.4 Alcance funcional 6
1.5 Usuarios y actores 7
1.6 Beneficios y características 7
1.7 Limitaciones conocidas 7
1.8 Estado actual 7
1.9 Cómo se clasifican las afirmaciones 7
Capítulo 2. Requisitos del sistema 8
2.1 Requisitos funcionales 8
2.2 Requisitos No funcionales 9
2.3 Reglas de negocio 9
2.4 Restricciones técnicas 10
2.5 Matriz de trazabilidad (requisito → módulo → evidencia) 10
Capítulo 3. Arquitectura de software 11
3.1 Visión general 11
3.2 Diagrama de contexto 11
3.3 Diagrama de contenedores 12
3.4 Diagrama de componentes del backend (verificado en bytecode) 13
3.5 Diagrama de componentes de la app Android (verificado) 13
3.6 Flujo de una solicitud HTTP 14
3.7 Organización de paquetes del backend 14
3.8 Decisiones arquitectónicas verificables 14
3.9 Dependencias entre componentes 14
Capítulo 4. Tecnologías y herramientas 15
4.1 Backend 15
4.2 App Android (v10) 15
4.3 Web 16
4.4 Riesgos de versiones y compatibilidad 16
Capítulo 5. Backend y Interfaz de programación REST 16
5.1 Estructura de paquetes 17
5.2 Componentes 17
5.3 Validaciones y manejo de errores 18

5.4 Autenticación y autorización reales 18
5.5 Configuración de Spring Boot y variables de entorno 18
5.6 Conexión a MySQL 19
5.7 Tabla de Puntos de acceso 19
5.8 Detalle de los Puntos de acceso verificados (versión A) 20
5.9 Diferencias entre versiones y contrato con la app 21
Capítulo 6. Base de datos 22
6.1 Motor y origen del esquema 22
6.2 Esquema (versión A, verificado en anotaciones) 22
6.3 Columnas adicionales de la versión B [Reportado] 23
6.4 Diagrama entidad–relación 24
6.5 Índices y restricciones identificados 24
6.6 DDL orientativo (No generado, No ejecutado) 25
6.7 Datos de prueba 25
6.8 Creación, migraciones, respaldos y restauración 25
6.9 Reglas de integridad de datos 26
Capítulo 7. Aplicaciones Android y web 26
7.1 Versiones disponibles 26
7.2 Android v10 26
7.3 Aplicación web [Reportado] 29
7.4 Diferencias entre versiones 29
Capítulo 8. Flujos, reglas y algoritmos 29
8.1 Ciclo de vida del pedido 29
8.2 Creación de un pedido 31
8.3 Asignación de repartidor 31
8.4 Confirmación por código de 4 dígitos 31
8.5 Cálculo del costo de envío (en la app) 31
8.6 Calificación 32
8.7 Inicio de sesión y deducción del rol 32
Capítulo 9. Seguridad y calidad 32
9.1 Hallazgos 32
9.2 Privacidad 33
9.3 Calidad y mantenibilidad 33
9.4 Prioridades antes de publicar 33
9.5 Matriz de permisos objetivo (propuesta, No es el comportamiento actual) 33
Capítulo 10. Pruebas y validación 34
10.1 Qué se hizo en este análisis 34
10.2 Pruebas existentes 34
10.3 Plan de pruebas recomendado 34
10.4 Verificación de rutas sin ejecutar 35
10.5 Comandos sugeridos 35

Capítulo 11. Instalación y configuración 35
11.1 Requisitos previos 35
11.2 Opción 1 — backend a partir del código fuente (`Proy1`) 35
11.3 Opción 2 — ejecutar las clases compiladas de `clases.rar` (solo verificación local) 36
11.4 App Android 36
11.5 Crear datos mínimos para probar (backend en marcha) 36
Capítulo 12. Despliegue y mantenimiento 36
12.1 Preparación para un entorno compartido 36
12.2 Construcción 36
12.3 App Android 36
12.4 Cambios de esquema 36
12.5 Mantenimiento de la documentación 36
12.6 Backlog técnico sugerido 37
Capítulo 13. Problemas frecuentes 37
Capítulo 14. Glosario 37
Capítulo 15. Fuentes y trazabilidad 38
Capítulo 16. Auditoría de la documentación previa 38
16.1 Correcciones confirmadas por el código 38
16.2 Contradicciones y vacíos que persisten 39
16.3 Defectos de forma del DOCX consolidado 40
16.4 Mejoras aplicadas en esta versión 40
16.5 Mejoras prioritarias pendientes (para cerrar el análisis) 40
Anexo — Inventario de archivos analizados 40
Archivos recibidos 40
Backend compilado (A) — 32 clases 41
Android v10 (C) 41
No recibidos 41
Hallazgos de higiene 41
Anexo — Limitaciones del análisis 42

Plataforma académica de pedidos y entregas de comida: **Interfaz de programación REST en Java/Spring Boot + app Android nativa**. Esta documentación reemplaza y corrige la versión anterior (`Documentacion_PideYa.zip` y `Documentacion_Completa_PideYa.docx`).

> **Base del análisis.** Backend **compilado** (`clases.rar`: 32 `.class`, leídos con `javap`) y app **Android v10** (código fuente). No se entregó el código fuente del backend (`Proy1`), ni `pom.xml`, `application.properties`, Gradle ni la web: lo que depende de ellos se marca **[Reportado]** (proviene del DOCX previo). Nada se compiló ni ejecutó.

Resumen

Aspecto	Resultado
Backend	com.deliveryapp.delivery_backend, Java 17, Spring Boot 3+ (JPA, Seguridad/BCrypt, MVC)
Roles	Cliente, Restaurante, Repartidor, Admin (una tabla usuarios, discriminador tipo_usuario)
Interfaz de programación	18 Puntos de acceso verificados + 9 reportados = 27; la app v10 usa 16
Base de datos	5 tablas: usuarios, productos, pedidos, detalle_pedido, calificaciones
App	com.pideya.app v10: cliente, comercio, repartidor; Retrofit + Gson; modo demostración sin servidor
Estado	Prototipo académico funcional; **No apto para producción**

Hallazgos principales

1. **El backend compilado es anterior a la app v10**: la app llama a 6 rutas (`/estado/{e}`, `/repartidor/{id}`, `/codigo`, `/aceptar`, `/rechazar`, `/entregar`) que ese backend No tiene, y envía campos que ignora (ubicación, pago, envío).
2. **Sin autenticación ni autorización** (`anyRequest().permitAll()`, CSRF off); cualquiera puede registrar administradores o modificar pedidos.
3. **Las respuestas incluyen el hash de contraseña**, incluso en `GET /Interfaz de programación/restaurantes` (público).
4. **El estado del pedido es texto libre** en el servidor; las reglas de transición solo viven en la app.
5. **Errores de negocio → HTTP 500**; sin validaciones de entrada.
6. La documentación previa tenía **12 imprecisiones** (p. ej. nombres de estado, “coordenadas fijas” en v10, modelo de herencia) corregidas en el Capítulo 16.

Contenido

Ver el índice. Capítulos: 1 Presentación · 2 Requisitos · 3 Arquitectura · 4 Tecnologías · 5 Backend y API · 6 Base de datos · 7 Android y web · 8 Flujos · 9 Seguridad · 10 Pruebas · 11 Instalación · 12 Despliegue · 13 Problemas · 14 Glosario · 15 Fuentes · 16 Auditoría.

Etiquetas de certeza

[Verificado] visto en bytecode/código entregado · **[Reportado]** solo en la documentación previa · **[Inferido]** deducción razonable · **[Pendiente]** requiere archivos o ejecución.

Índice de documentación

Cap.	Documento	Contenido
1	Presentación del proyecto	Qué es, objetivos, alcance, actores, limitaciones, estado
2	Requisitos del sistema	RF/RNF, reglas de negocio, matriz de trazabilidad
3	Arquitectura de software	Contexto, contenedores, componentes, flujo HTTP, decisiones
4	Tecnologías y herramientas	Stack, tems iz comprobadas, compatibilidad
5	Backend y Interfaz de programación REST	Paquetes, Puntos de acceso (27), ejemplos curl, contrato con la app
6	Base de datos	Esquema, ER, DDL orientativo, integridad
7	Aplicaciones Android y web	Pantallas, navegación, comunicación, hallazgos
8	Flujos, reglas y algoritmos	Estados, pedido, tems, código
9	Seguridad y calidad	Hallazgos S-01…S-16, matriz de permisos

Cap.	Documento	Contenido
10	Pruebas y validación	Qué se hizo y plan de pruebas
11	Instalación y configuración	Backend, app, datos mínimos
12	Despliegue y mantenimiento	Producción, backlog
13	Problemas frecuentes	Diagnóstico
14	Glosario	Términos
15	Fuentes y trazabilidad	Origen de cada dato
16	Auditoría de la documentación previa	Correcciones, vacíos, mejoras
—	Inventario de archivos analizados · Limitaciones del análisis	Qué se revisó realmente
—	Catálogo de diagramas	Imágenes y fuentes Mermaid

Capítulo 1. Presentación del temsiz

5.5 Nombre y descripción

PideYa es un prototipo académico de plataforma de pedidos y entrega de comida (delivery). Conecta tres actores — cliente, restaurante y repartidor— mediante una **Interfaz de programación REST en Java/Spring Boot** y una **aplicación Android nativa**. Los importes se muestran en quetzales (**Q**) y la ubicación por defecto es Ciudad de Guatemala (14.6349, −90.5069), lo que sitúa el proyecto en Guatemala. [Verificado: `Ui.java`, `Ubicación.java`]

1.2 Problema que busca resolver

Coordinar un pedido exige registrar al cliente, elegir comercio y productos, calcular importes, indicar dónde entregar, seguir el estado y asociar un repartidor. PideYa agrupa esas operaciones en un backend común que distintos clientes (Android y web) consumen por HTTP/JSON.

1.3 Objetivos

General. Digitalizar el ciclo completo de un pedido de comida: explorar → pedir → preparar → entregar → calificar.

Específicos (derivados del código y del documento de requisitos citado en la documentación previa; No se recibió el documento de requisitos original):

1. Registrar usuarios por rol e iniciar sesión.
2. Consultar restaurantes y sus menús.
3. Crear pedidos con varios productos y total calculado por el servidor.
4. Hacer seguimiento del estado del pedido y asignar repartidor.
5. Confirmar la entrega mediante un código de 4 dígitos.
6. Calificar pedidos entregados (una vez por pedido).
7. Ofrecer una app móvil por rol (cliente, comercio, repartidor).
8. Persistir los datos en una base relacional.

1.4 Alcance temsizer

Capacidad	Estado	Detalle
Registro / login	Implementado	Contraseñas con Bcrypt
Catálogo de restaurantes y productos	Implementado	Solo restaurantes activo aparecen en el listado
Pedidos con detalle y total	Implementado	Total = Σ precio × cantidad
Estados, asignación de repartidor	Parcial	El servidor No valida las transiciones (versión compilada)

Capacidad	Estado	Detalle
Ubicación, pago, envío, código de entrega	Parcial	Presentes en la app v10 y en el backend fuente (según documentación previa); ausentes del backend compilado
Aprobación de cuentas	Reportado	Solo en el backend fuente (CuentaController), No verificable aquí
Pagos con tarjeta reales	No implementado	Solo se guarda el texto “TARJETA”/“EFECTIVO”
Asignación automática por cercanía	No implementado	La asignación es manual por ID
Panel de administración	No implementado	Solo existe el registro de administradores
Descuentos y promociones	No encontrado	—

1.5 Usuarios y tems

Actor	Rol en el backend	App Android v10	Pantalla inicial
Cliente	CLIENTE	Sí	RestaurantesActivity
Restaurante / comercio	RESTAURANTE	Sí	PedidosComercioActivity
Repartidor	REPARTIDOR	Sí	EntregasActivity
Administrador	ADMIN	No (la app lo rechaza con un mensaje)	—

1.6 Beneficios y características

- Arquitectura por capas clara (controlador → servicio → repositorio → entidad).
- **POO aplicada:** `Usuario` abstracta con `obtenerPanel()` en el backend y `pantallaInicial()` en Android (polimorfismo).
- App desacoplada del origen de datos: `DeliveryRepository` con dos implementaciones (`ApiRepository`, `MockRepository`).
- Modo demostración sin servidor, tema claro/oscuro y animaciones.
- Costo de envío estimado por distancia y apertura de la ubicación en Google Maps.

1.7 Limitaciones conocidas

1. No hay autenticación ni autorización: toda ruta está permitida.
2. Las respuestas JSON incluyen el hash de contraseña de los usuarios.
3. El estado del pedido es texto libre en el backend compilado.
4. Los errores de negocio devuelven HTTP 500.
5. Existen al menos **dos versiones de backend** (compilada y fuente) con contratos distintos.
6. No hay migraciones SQL ni pruebas funcionales.

1.8 Estado actual

Prototipo académico funcional, No apto para producción. El análisis fue estático (lectura de bytecode y de código fuente Android). No se compiló, ejecutó ni desplegó nada. Ver Capítulo 10 y Limitaciones del análisis.

1.9 Cómo se clasifican las afirmaciones

Etiqueta	Significado
[Verificado]	Observado directamente en el bytecode del backend compilado (fuente A) o en el código/recursos Android (fuente C)
[Reportado]	Procede del documento consolidado previo (fuente B: backend fuente Proy1, web, PDF), cuyos archivos **No se entregaron** en este análisis
[Inferido]	Deducción razonable; se indica en qué se basa

Etiqueta	Significado
[Pendiente]	Requiere ejecutar o recibir archivos adicionales

Capítulo 2. Requisitos del sistema

Estados usados: Correcto: Implementado · Parcial · Descripción: Propuesto · Pendiente de verificar. **Fuentes: A** = backend compilado (`clases.rar`), **B** = backend fuente `Proy1` según documentación previa, **C** = app Android v10.

2.1 Requisitos funcionales

ID	Requisito	Estado	A	B	C	Evidencia
RF-01	Registrar clientes	Correcto:	Correcto:	Correcto:	Correcto:	UsuarioController.registrar, RegistroActivity
RF-02	Registrar restaurantes, repartidores y administradores		Correcto:	Correcto:	Incorrecto:	RestauranteController, RepartidorController, AdminController; la app solo registra clientes
RF-03	Iniciar sesión con correo y contraseña	Correcto:	Correcto:	Correcto:	Correcto:	UsuarioService.login, LoginActivity
RF-04	Listar restaurantes activos	Correcto:	Correcto:	Correcto:	Correcto:	UsuarioService.listarRestaurantes filtra Restaurante::isActivo
RF-05	Crear, listar, editar y eliminar productos	Correcto:	Correcto:	Correcto:		ProductoController; la app solo lista el menú
RF-06	Crear un pedido con varios productos	Correcto:	Correcto:	Correcto:	Correcto:	PedidoService.crearPedido, MenuActivity
RF-07	Calcular el total del pedido en el servidor	Correcto:	Correcto:		—	Σ precio × cantidad en crearPedido
RF-08	Consultar pedidos por cliente	Correcto:	Correcto:	Correcto:	Correcto:	GET /Interfaz de programación/pedidos/cliente/{id}
RF-09	Consultar pedidos por restaurante	Correcto:	Correcto:	Correcto:	Correcto:	GET /Interfaz de programación/pedidos/restaurante/{id}
RF-10	Consultar pedidos por repartidor y por estado		Incorrecto:	Correcto:	Correcto:	La app los invoca; No existen en A
RF-11	Cambiar el estado del pedido		Correcto: (sin reglas)		Correcto: (reglas en cliente)	actualizarEstado, EstadoPedido.puedePasarA
RF-12	Aceptar y rechazar pedidos (restaurante)		Incorrecto:	Correcto:	Correcto:	PUT .../aceptar, .../rechazar
RF-13	Asignar repartidor a un pedido		Correcto:	Correcto:	Correcto:	En A fuerza EN_CAMINO; la app espera que el pedido esté LISTO
RF-14	Guardar ubicación de entrega (lat/lng)		Incorrecto:	Correcto:	Correcto:	PedidoRequest de B; la app envía latitudEntrega/longitudEntrega
RF-15	Registrar método de pago (efectivo/tarjeta) y costo de envío		Incorrecto:	Correcto:	Correcto:	Solo etiqueta; sin pasarela de pago
RF-16	Confirmar la entrega con código de 4 dígitos		Incorrecto:	Correcto:	Correcto:	GET .../codigo, PUT .../entregar
RF-17	Calificar un pedido entregado (1 vez)	Correcto:	Correcto:	Correcto:	Correcto:	CalificacionService; pedido_id único
RF-18	Aprobar cuentas pendientes y cambiar disponibilidad de repartidor		Incorrecto:	Correcto:	Incorrecto:	CuentaController (solo en B)

ID	Requisito	Estado	A	B	C	Evidencia
RF-19	Mostrar la ubicación de entrega en Google Maps	Correcto:	—	—	Correcto:	Ui.verEnGoogleMaps (intent externo, No mapa embebido)
RF-20	Estimar el costo de envío por distancia	Correcto:	Incorrecto:		Correcto:	Ui.costoEnvio (cálculo en el cliente)
RF-21	Modo oscuro y recordar el tema	Correcto:	—	—	Correcto:	Ui.alternarTema
RF-22	Ejecutar la app sin servidor (modo demostración)	Correcto:	—	—	Correcto:	Repo.USAR_BACKEND, MockRepository
RF-23	Asignación automática al repartidor más cercano	Descripción:	Incorrecto:	Incorrecto:	Incorrecto:	Repartidor No almacena coordenadas
RF-24	Panel de administración y estadísticas	Descripción:	Incorrecto:	Incorrecto:	Incorrecto:	Solo el texto de Admin.obtenerPanel()
RF-25	Pagos con tarjeta mediante pasarela	Descripción:	Incorrecto:	Incorrecto:	Incorrecto:	No hay integración
RF-26	Descuentos y promociones	Descripción:	Incorrecto:	Incorrecto:	Incorrecto:	No hay entidades ni rutas

2.2 Requisitos No funcionales

ID	Categoría	Requisito	Estado	Evidencia
RNF-01	Seguridad	Contraseñas almacenadas con hash BCrypt	Correcto: (A)	SecurityConfig.passwordEncoder() → BcryptPasswordEncoder; registrar* llama a encode
RNF-02	Seguridad	Autenticación de cada solicitud y autorización por rol	Incorrecto:	authorizeHttpRequests(anyRequest().permitAll()), CSRF desactivado
RNF-03	Seguridad	No exponer datos internos en las respuestas	Incorrecto:	Usuario.password No está ignorado en JSON
RNF-04	Seguridad	tems izer tránsito	Incorrecto:	usesCleartextTraffic="true", URL http://
RNF-05	Usabilidad	Mensajes de error comprensibles en español		La app traduce los errores HTTP a mensajes propios
RNF-06	Usabilidad	Interfaz con tema claro/oscuro y animaciones	Correcto:	values/ y values-night/, carpeta anim/
RNF-07	Mantenibilidad	Capas separadas y repositorio intercambiable	Correcto:	Patrón repositorio en la app; capas en el backend
RNF-08	Mantenibilidad	Pruebas automatizadas	Incorrecto:	Sin pruebas conocidas salvo contextLoads() (según B)
RNF-09	Integridad	Unicidad de correo y de calificación por pedido	Correcto:	unique=true y comprobación en servicio
RNF-10	Disponibilidad / rendimiento	Sin tems izer ni medidas		—
RNF-11	Portabilidad	Android con permisos de ubicación e Internet	Correcto:	AndroidManifest.xml

2.3 Reglas de negocio

ID	Regla	Dónde se aplica	Fuente
RN-01	El correo es único entre todos los roles	Servicio (findByEmail) y columna unique	A

ID	Regla	Dónde se aplica	Fuente
RN-02	La contraseña se guarda con BCrypt	UsuarioService.registrar*	A
RN-03	Solo los restaurantes activo = true se listan; el valor por defecto es true	listarRestaurantes, constructor de Restaurante	A
RN-04	Un repartidor nuevo está disponible = true; un producto nuevo, disponible = true	Constructores	A
RN-05	Un pedido nuevo nace en PENDIENTE	Constructor de Pedido	A
RN-06	subtotal = precio del producto × cantidad; total = Σ subtotales	crearPedido	A
RN-07	Asignar un repartidor deja el pedido en EN_CAMINO	asignarRepartidor	A
RN-08	Solo se califican pedidos ENTREGADO, una sola vez	CalificacionService	A
RN-09	Transiciones válidas: PENDIENTE→ACEPTADO/RECHAZADO/CANCELADO; ACEPTADO→PREPARANDO/CANCELADO; PREPARANDO→LISTO; LISTO→EN_CAMINO; EN_CAMINO→ENTREGADO	EstadoPedido.puedePasarA (cliente); el servidor No las aplica en A	C
RN-10	Costo de envío = Q8 + Q2.50 × km, máximo Q60; tarifa fija Q15 si el restaurante no tiene coordenadas	Ui.costoEnvio	C
RN-11	Correo con formato válido y contraseña de ≥ 5 caracteres	RegistroActivity (solo en el cliente)	C
RN-12	La app solo atiende clientes, repartidores y comercios	ApiRepository.usuario	C
RN-13	Solo un pedido LISTO puede ser tomado por un repartidor (en la app)	EntregasActivity, MockRepository	C

2.4 Restricciones técnicas

- Backend: Java 17 (bytecode tems i 61), Spring Boot con `tems i.*` (Spring Boot 3 o superior). [Verificado]
- Base de datos relacional con JPA; MySQL según documentación previa. [Reportado]
- Android con permisos `INTERNET`, `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION` y Google Play Services Location. [Verificado]
- Identificadores `Long` en el servidor; `int` en la app (suficiente en un prototipo, un límite en producción).

2.5 Matriz de trazabilidad (requisito → módulo → evidencia)

Requisito	Módulo backend (A)	Módulo Android ©	Evidencia verificable
RF-01, RF-03	UsuarioController, UsuarioService, UsuarioRepository	LoginActivity, RegistroActivity, ApiRepository.login/registrar	Anotaciones @PostMapping(“/registro”), @PostMapping(“/login”)
RF-04	RestauranteController.listar, UsuarioService.listarRestaurantes	RestaurantesActivity	instanceof Restaurante + isActivo en bytecode
RF-05	ProductoController, ProductoService	MenuActivity (lista)	4 rutas bajo /Interfaz de programación/productos
RF-06, RF-07	PedidoController.crear, PedidoService.crearPedido	MenuActivity.mostrarResumen, ApiRepository.crearPedido	Bytecode: dmul + dadd
RF-08, RF-09	PedidoController, PedidoRepository.findByClienteId/RestauranteId	PedidosActivity, PedidosComercioActivity	Rutas /cliente/{id}, /restaurante/{id}
RF-11, RF-13	PedidoService.actualizarEstado/asignarRepartidor	EstadoPedido, EntregasActivity	ldc “EN_CAMINO”

Requisito	Módulo backend (A)	Módulo Android ©	Evidencia verificable
RF-17	CalificacionService, CalificacionRepository.findByPedidoId	PedidosActivity.calificar	Mensajes de error en bytecode
RF-12, RF-16	— (solo en B)	ApiRepository (rutas aceptar, rechazar, entregar, tems)	Interfaz Interfaz de programación de Retrofit
RF-14, RF-15, RF-20	— (solo en B)	Ubicacion, Entrega, Ui.costoEnvio	Campos enviados en crearPedido
RF-02, RF-18	RestauranteController, RepartidorController, AdminController (+ CuentaController en B)	—	Rutas /registro

Capítulo 3. Arquitectura de software

3.1 Visión general

PideYa es un sistema **cliente–servidor de tres niveles**:

1. **Clientes:** app Android (verificada) e interfaz web HTML/JS (reportada, No entregada).
2. **Servidor de aplicación:** Interfaz de programación REST sin estado (Spring Boot).
3. **Persistencia:** base relacional accedida mediante JPA/Hibernate.

El backend sigue la separación clásica **controlador → servicio → repositorio → entidad**. La app sigue un patrón **repositorio** (`DeliveryRepository`) que aísla las pantallas del origen de datos.

3.2 Diagrama de temsi

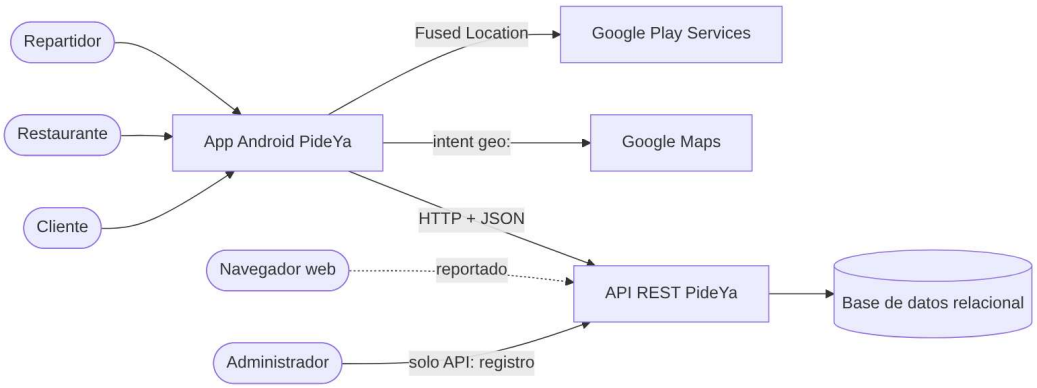


Diagrama 03.1

3.3 Diagrama de contenedores

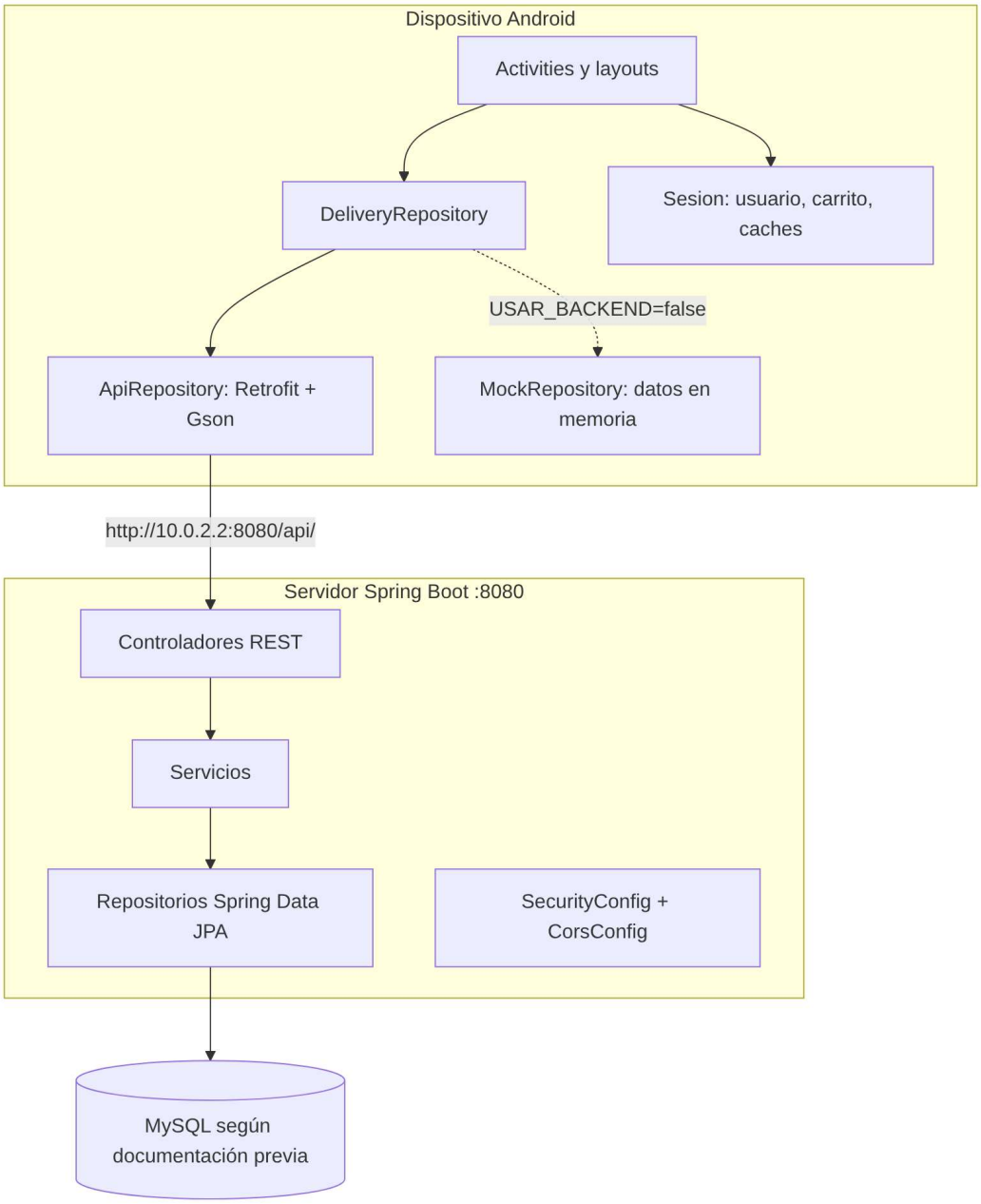


Diagrama 03.2

3.4 Diagrama de componentes del backend (verificado en bytecode)

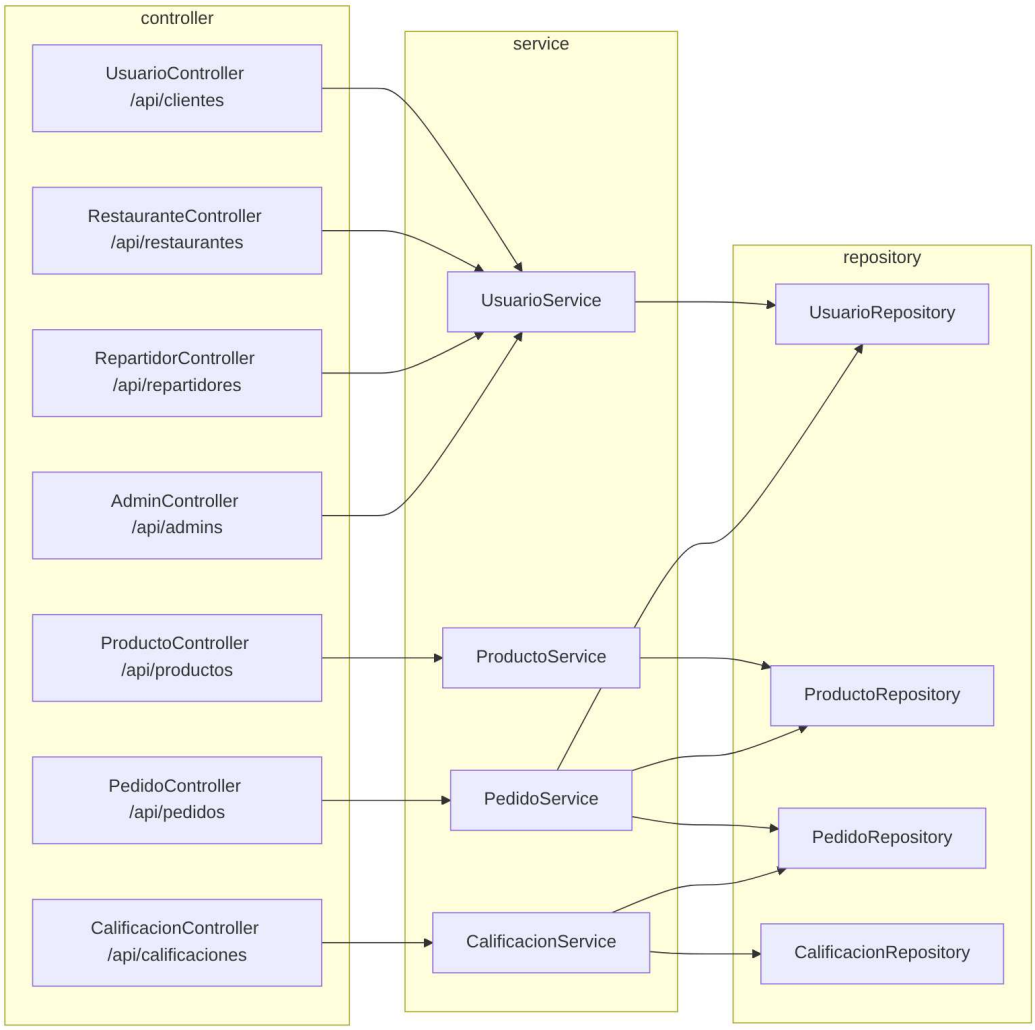


Diagrama 03.3

3.5 Diagrama de componentes de la app Android (verificado)

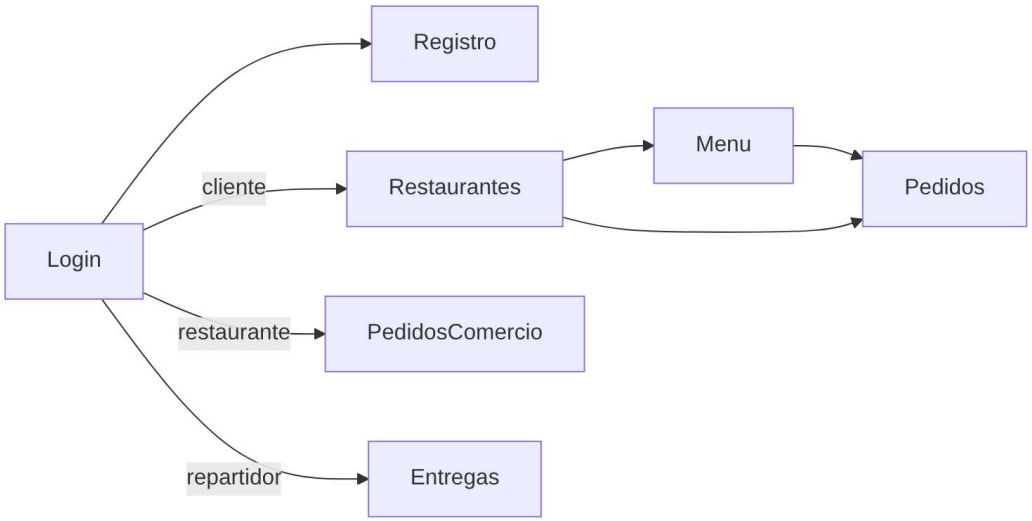


Diagrama 03.4

3.6 Flujo de una solicitud HTTP

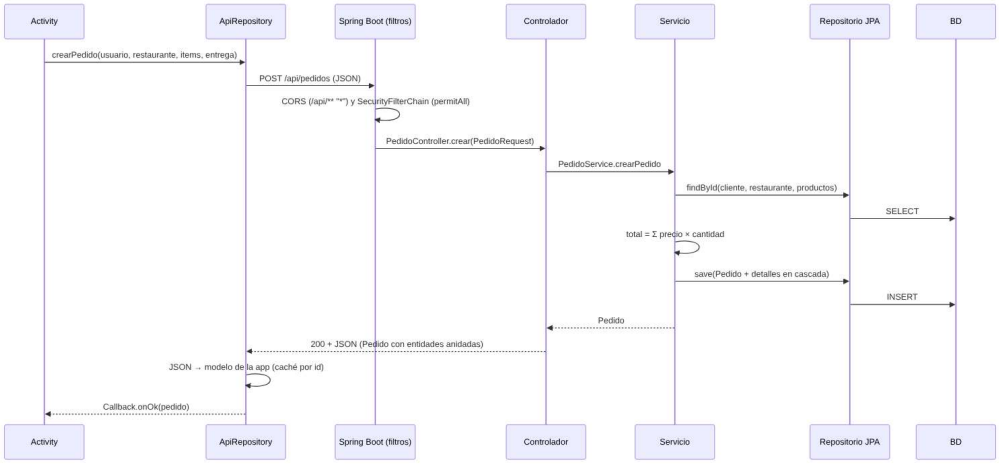


Diagrama 03.5

Si un servicio lanza `RuntimeException` (por ejemplo, "Cliente No encontrado"), No hay `@ControllerAdvice` en el backend compilado: Spring responde **HTTP 500** y la app muestra su mensaje genérico. [Verificado: ausencia de clase manejadora; comportamiento 500 inferido del comportamiento por defecto de Spring]

3.7 Organización de paquetes del backend

Paquete raíz `com.deliveryapp.delivery_backend` (ver estructura completa).

Paquete	Responsabilidad
config	SecurityConfig (Bcrypt, CSRF off, permitAll), CorsConfig
controller	7 controladores REST (en B: 8, con CuentaController)
service	Reglas de negocio: usuarios, productos, pedidos, calificaciones
repository	Interfaces JpaRepository con consultas derivadas
model	Entidades JPA y objetos de petición (LoginRequest, PedidoRequest, CalificacionRequest)

3.8 Decisiones arquitectónicas verificables

Decisión	Evidencia	Consecuencia
Herencia JPA SINGLE_TABLE con discriminador tipo_usuario	@Inheritance, @DiscriminatorColumn en Usuario	Una sola tabla usuarios; correo único global; las columnas de subclases quedan nulas para otros roles
Entidades JPA expuestas directamente como respuesta	Controladores devuelven Pedido, Usuario, etc.	Fuga de password (hash) y acoplamiento API ↔ esquema
Estado del pedido como String	Pedido.estado: String	Sin validación de valores; riesgo de inconsistencias
Validaciones con RuntimeException	Bytecode de los servicios	Respuestas 500 en lugar de 400/404/409
@JsonIgnore en DetallePedido.pedido	Anotación en bytecode	Evita la recursión Pedido ↔ DetallePedido
Patrón repositorio en Android con interruptor USAR_BACKEND	Repo.java	Se puede demostrar sin servidor; riesgo de publicar con datos ficticios
Rol del usuario deducido por los campos del JSON	ApiRepository.usuario	Dependencia frágil del formato de la respuesta (ver Cap. 7)
Sesión en memoria estática (Sesion)	Sesion.java	Se pierde al cerrar la app; sin tokens

3.9 Dependencias entre temsizer

- `PedidoService` depende de tres repositorios (`Pedido`, `Producto`, `Usuario`); No usa `ProductoService`.

- `CalificacionService` depende de `CalificacionRepository` y `PedidoRepository`.
- `UsuarioService` es compartido por cuatro controladores (clientes, restaurantes, repartidores, admins).
- La app depende del backend solo a través de la interfaz `Interfaz de programación` de `ApiRepository` (16 operaciones).

Capítulo 4. Tecnologías y herramientas

Sobre las versiones. El backend compilado No incluye `pom.xml` y la app No incluye Gradle, por lo que **No es posible comprobar versiones de librerías** desde los archivos entregados. Las versiones que aparecen proceden del bytecode (Java) o de la documentación previa (marcadas **[Reportado]**).

4.1 Backend

Tecnología	Versión	Función	Dónde se usa	Evidencia
Java	17 (bytecode 61)	Lenguaje y JVM mínima	Todo el backend	[Verificado] major version: 61
Spring Boot	4.1.1 según pom.xml previo	Arranque, autoconfiguración	DeliveryBackendApplication (@SpringBootApplication)	[Reportado] tems i; [Verificado] uso
Spring MVC	con Boot	Interfaz de programación REST	@RestController, @RequestMapping, @GetMapping...	[Verificado]
Spring Data JPA + Hibernate	con Boot	Persistencia ORM	repository/*, entidades tems i.persistence.*	[Verificado]
Jakarta Persistence	3.x (tems i.*)	Anotaciones de entidades	model/*	[Verificado] el espacio de nombres exige Boot 3+
Spring Seguridad (crypto + web)	con Boot	BCrypt y cadena de filtros	SecurityConfig	[Verificado]
Jackson	con Boot	JSON; @JsonIgnore	DetallePedido	[Verificado]
Lombok	—	Genera getters/setters/equals/toString	Entidades y DTO (métodos presentes en bytecode sin fuente)	[Inferido]
MySQL + Connector/J	—	Base de datos	application.properties (No entregado)	[Reportado]
Maven + Wrapper	—	Construcción	pom.xml, mvnw (No entregados)	[Reportado]

Compatibilidad: Java 17 o superior. Con Spring Boot 4.x se recomienda verificar que la versión exista en el repositorio Maven configurado antes de compilar (versión reportada, No comprobada).

4.2 App Android (v10)

Tecnología	Función	Dónde se usa	Evidencia
Java (lenguaje de la app)	Código de la app	app/src/main/java	[Verificado] 26 archivos .java
Android SDK / AppCompat	Actividades, diálogos	Todas las *Activity	[Verificado] imports
Material 3 DayNight	Tema claro/oscuro	Theme.PideYaApp (Theme.Material3.DayNight.NoActionBar)	[Verificado] pideya_theme.xml
RecyclerView	Listas	ListaAdapter	[Verificado]
Retrofit 2 + converter-gson	Cliente HTTP	ApiRepository	[Verificado] imports
OkHttp 3	RequestBody, MediaType	ApiRepository	[Verificado]

Tecnología	Función	Dónde se usa	Evidencia
Gson	JSON dinámico (JsonObject)	ApiRepository	[Verificado]
Google Play Services Location	Ubicación (FusedLocationProviderClient)	Ubicacion.actualizar	[Verificado]
LocationManager (Android)	Última ubicación conocida	Ubicacion.actual	[Verificado]
Activity Result Interfaz de programación	Solicitud de permisos	RestaurantesActivity, MenuActivity	[Verificado]
Google Maps (app externa)	Ver la entrega	Ui.verEnGoogleMaps (geo:, con respaldo web)	[Verificado] No usa SDK de mapas

No determinables: `compileSdk`, `minSdk`, versión de Gradle/AGP y versiones de las librerías (No hay archivos Gradle). Sin ellos **la app No puede compilarse** con solo este ZIP.

4.3 Web

HTML + JavaScript con `fetch` hacia `/api/clientes/*`. [Reportado; los archivos No se entregaron]

4.4 Riesgos de temsiz y compatibilidad

Riesgo	Detalle	Estado
Versión de Spring Boot muy reciente	Reportada 4.1.1; confirmar disponibilidad y compatibilidad de las dependencias	[Pendiente]
usesCleartextTraffic=”true”	Android 9+ bloquea HTTP por defecto; aquí se habilita explícitamente	[Verificado]
RequestBody.Crear(MediaType, String)	Firma antigua de OkHttp ¾; en OkHttp 5 está retirada	[Inferido] depende de la versión real
Identificadores int en la app	Los Ids del servidor son Long	[Verificado]
Importes en Double	Riesgo de redondeo; se recomienda BigDecimal	[Verificado]

Capítulo 5. Backend y Interfaz de programación REST

Dos versiones de backend. Este capítulo documenta con detalle la **versión A** (backend compilado de `clases.rar`, 32 archivos `.class`, leído con `javap`; [Verificado]). Lo que solo existe en la **versión B** (código fuente `Proy1`, descrito en la documentación consolidada previa) se marca **[Reportado]** porque esos archivos No se entregaron. La app Android v10 (fuente C) se ajusta a B, No a A.

5.1 Estructura de paquetes

```
com.deliveryapp.delivery_backend
├── DeliveryBackendApplication        @SpringBootApplication, main()
├── config
│   ├── SecurityConfig                BcryptPasswordEncoder + SecurityFilterChain (CSRF off, permitAll)
│   └── CorsConfig (+ clase anónima)  CORS /api/** → origen "*", GET POST PUT DELETE
├── controller
│   ├── UsuarioController             /api/clientes
│   ├── RestauranteController         /api/restaurantes
│   ├── RepartidorController          /api/repartidores
│   ├── AdminController               /api/admins
│   ├── ProductoController            /api/productos
│   ├── PedidoController              /api/pedidos
│   └── CalificacionController        /api/calificaciones
│   └── (CuentaController)            /api/aprobaciones … solo en B [Reportado]
├── service
│   ├── UsuarioService   ProductoService   PedidoService   CalificacionService
├── repository
│   ├── UsuarioRepository  ProductoRepository  PedidoRepository  CalificacionRepository
└── model
    ├── Usuario (abstracta) ← Cliente, Restaurante, Repartidor, Admin
    ├── Producto, Pedido, DetallePedido, Calificacion
    └── LoginRequest, PedidoRequest (+ ItemPedido), CalificacionRequest
```

5.2 Componentes

Controladores

Todos usan `@RestController` e inyectan su servicio con `@Autowired` (inyección por campo). No hay `@CrossOrigin` ni `@Valid`.

Servicios (lógica de negocio)

Servicio	Métodos	Reglas
UsuarioService	registrarCliente/Restaurante/Repartidor/Admin, buscarPorEmail, login, listarRestaurantes	Correo duplicado → error; BCrypt en registro; login compara con matches; el listado filtra instanceof Restaurante y isActivo
ProductoService	crear, listarPorRestaurante, tems izer, eliminar	actualizar copia nombre, descripción, precio y disponibilidad; "Producto No encontrado" si No existe. No valida propiedad
PedidoService	crearPedido, listarPorCliente, listarPorRestaurante, listarPendientes, buscarPorId, actualizarEstado, asignarRepartidor	Ver Cap. 8
CalificacionService	calificar	Solo ENTREGADO; una por pedido

Repositorios (`JpaRepository<Entidad, Long>`)

Repositorio	Consultas derivadas
UsuarioRepository	findByEmail → Optional<Usuario>
ProductoRepository	findByRestauranteId → List<Producto>
PedidoRepository	findByClienteId, findByRestauranteId, findByEstado (String)
CalificacionRepository	findByPedidoId → Optional<Calificacion>

Entidades y DTO

Ver Capítulo 6. Los DTO son `LoginRequest {email, password}`, `PedidoRequest {clienteId, restauranteId,  tems[{productoId, cantidad}]}` y `CalificacionRequest {estrellas, comentario}`. No tienen anotaciones de validación.

5.3 Validaciones y manejo de errores

- **No hay Bean Validation** (`@Valid`, `@NotNull`…) ni `@ControllerAdvice`. [Verificado]
- Todas las validaciones de negocio lanzan `RuntimeException` con estos mensajes:

Mensaje	Origen
“Ya existe un usuario con ese email”	registrar*
“Usuario No encontrado”	buscarPorEmail (login)
“Contraseña incorrecta”	login
“Cliente / Restaurante / Producto No encontrado”	crearPedido
“Pedido No encontrado”, “Repartidor No encontrado”	PedidoService
“Producto No encontrado”	ProductoService.actualizar
“Solo se pueden calificar pedidos entregados”, “Este pedido ya fue calificado”, “Pedido No encontrado”	CalificacionService

- **Resultado:** Spring devuelve **HTTP 500** y, por defecto, No incluye el mensaje en el cuerpo. [Inferido: comportamiento por defecto de Spring Boot sin manejador]
- Faltan: validación de cantidad > 0, de `tems` vacío o nulo, de rango de estrellas (1–5), de estados permitidos y de pertenencia del producto al restaurante.
- `login` revela si el correo existe (“Usuario No encontrado” vs “Contraseña incorrecta”), lo que permite **enumerar cuentas**.

5.4 Autenticación y autorización reales

```
// Reconstrucción fiel del bytecode de SecurityConfig.filterChain
http.csrf(csrf -> csrf.disable())
    .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
```

- No hay login con sesión ni JWT: `POST /Interfaz de programación/clientes/login` solo **devuelve el objeto** `Usuario`. La app lo guarda en memoria y No envía credenciales en solicitudes posteriores.
- No hay roles aplicados: cualquier cliente HTTP puede registrar administradores, borrar productos o cambiar el estado de cualquier pedido.
- CORS: `addMapping(“/api/**”)`, `allowedOrigins("*")`, métodos `GET, POST, PUT, Eliminar`.

5.5 Configuración de Spring Boot y variables de entorno

`application.properties` **No se entregó**. Según la documentación previa [Reportado] contiene: URL MySQL local (`delivery_db`, con `createDatabaseIfNotExist=true`), usuario y **contraseña en texto plano** (No se reproduce), `spring.jpa.hibernate.ddl-auto=Actualizar` y `show-sql=true`.

Plantilla recomendada (nombres propuestos, No verificados contra el proyecto):

```
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/delivery_db}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
```

Variable	Obligatoria	Uso
DB_URL	No (valor por defecto local)	Cadena JDBC
DB_USER	Sí	Usuario de BD con privilegios limitados
DB_PASSWORD	Sí	Contraseña; nunca en el repositorio

5.6 Conexión a MySQL

[Reportado] Driver MySQL Connector/J, base `delivery_db` en `localhost:3306`. En el bytecode solo se observa JPA genérico (`jakarta.persistence`); No hay referencias directas a MySQL.

5.7 Tabla de Puntos de acceso

Leyenda: A = backend compilado [Verificado]; B = backend fuente [Reportado]; App = usado por Android v10 [Verificado en `ApiRepository.Interfaz de programación`]. Todos son públicos (`permitAll`).

#	Método	Ruta	Propósito	A	B	App
1	POST	/Interfaz de programación/clientes/registro	Registrar cliente	Correcto:	Correcto:	Correcto:
2	POST	/Interfaz de programación/clientes/login	Iniciar sesión (cualquier rol)	Correcto:	Correcto:	Correcto:
3	POST	/Interfaz de programación/restaurantes/registro	Registrar restaurante	Correcto:	Correcto:	—
4	GET	/Interfaz de programación/restaurantes	Listar restaurantes activos	Correcto:	Correcto:	Correcto:
5	POST	/Interfaz de programación/repartidores/registro	Registrar repartidor	Correcto:	Correcto:	—
6	PUT	/Interfaz de programación/repartidores/{id}/disponible	Cambiar disponibilidad	Incorrecto:	Correcto:	—
7	POST	/Interfaz de programación/admins/registro	Registrar administrador	Correcto:	Correcto:	—
8	GET	/Interfaz de programación/aprobaciones/pendientes	Listar cuentas pendientes	Incorrecto:	Correcto:	—
9	PUT	/Interfaz de programación/aprobaciones/{id}	Aprobar cuenta	Incorrecto:	Correcto:	—
10	POST	/Interfaz de programación/productos	Crear producto	Correcto:	Correcto:	—
11	GET	/Interfaz de programación/productos/restaurante/{restauranteId}	Menú de un restaurante	Correcto:	Correcto:	Correcto:
12	PUT	/Interfaz de programación/productos/{id}	Actualizar producto	Correcto:	Correcto:	—
13	Eliminar	/Interfaz de programación/productos/{id}	Eliminar producto	Correcto:	Correcto:	—
14	POST	/Interfaz de programación/pedidos	Crear pedido	Correcto:	Correcto:	Correcto:
15	GET	/Interfaz de programación/pedidos/{pedidoId}	Pedido por ID	Correcto:	Correcto:	—
16	GET	/Interfaz de programación/pedidos/pendientes	Pedidos en estado PENDIENTE	Correcto:	Correcto:	—
17	GET	/Interfaz de programación/pedidos/cliente/{clienteId}	Pedidos de un cliente	Correcto:	Correcto:	Correcto:
18	GET	/Interfaz de programación/pedidos/restaurante/{restauranteId}	Pedidos de un restaurante	Correcto:	Correcto:	Correcto:
19	GET	/Interfaz de programación/pedidos/repartidor/{repartidorId}	Pedidos de un repartidor	Incorrecto:	Correcto:	Correcto:
20	GET	/Interfaz de programación/pedidos/estado/{estado}	Pedidos por estado	Incorrecto:	Correcto:	Correcto:
21	PUT	/Interfaz de programación/pedidos/{pedidoId}/estado	Cambiar estado (cuerpo de texto)	Correcto:	Correcto:	Correcto:
22	PUT	/Interfaz de programación/pedidos/{pedidoId}/repartidor/{repartidorId}	Asignar repartidor	Correcto:	Correcto:	Correcto:
23	PUT	/Interfaz de programación/pedidos/{pedidoId}/aceptar	Aceptar pedido	Incorrecto:	Correcto:	Correcto:
24	PUT	/Interfaz de programación/pedidos/{pedidoId}/rechazar	Rechazar pedido	Incorrecto:	Correcto:	Correcto:
25	GET	/Interfaz de programación/pedidos/{pedidoId}/codigo	Consultar código de entrega	Incorrecto:	Correcto:	Correcto:
26	PUT	/Interfaz de programación/pedidos/{pedidoId}/entregar	Confirmar entrega con código	Incorrecto:	Correcto:	Correcto:
27	POST	/Interfaz de programación/calificaciones/pedido/{pedidoId}	Calificar pedido	Correcto:	Correcto:	Correcto:

Totales: **A = 18**, **B = 27**, **usados por la app = 16**. Las rutas 19, 20 y 23–26 son invocadas por la app pero **No existen en A**: con ese backend, las pantallas de repartidor y de aceptación del comercio fallarían (ver §5.9).

5.8 Detalle de los Puntos de acceso verificados (versión A)

> Todos devuelven **HTTP 200** cuando tienen éxito (los métodos devuelven la entidad o lista directamente). Los errores de negocio producen **500**. Los ejemplos usan `localhost:8080`; los IDs son ilustrativos.

Clientes

```
# Registro (devuelve el Cliente, incluido el hash de contraseña)
curl -X POST http://localhost:8080/Interfaz de programación/clientes/registro \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Ana","email":"ana@correo.com","password":"secreto123","direccionFavorita":"Zona 10"}'

# Login
curl -X POST http://localhost:8080/Interfaz de programación/clientes/login \
  -H "Content-Type: application/json" \
  -d '{"email":"ana@correo.com","password":"secreto123"}'
```

Errores: correo duplicado, “Usuario No encontrado”, “Contraseña incorrecta” (todos 500).

Restaurantes, repartidores y administradores

```
curl -X POST http://localhost:8080/Interfaz de programación/restaurantes/registro -H "Content-Type:
application/json" \
  -d '{"nombre":"Luis","email":"pizza@correo.com","password":"secreto123","nombreComercial":"Pizza
Roma","direccion":"Zona 4"}'

curl -X POST http://localhost:8080/Interfaz de programación/repartidores/registro -H "Content-Type:
application/json" \
  -d '{"nombre":"Marco","email":"marco@correo.com","password":"secreto123","vehiculo":"Moto"}'

curl -X POST http://localhost:8080/Interfaz de programación/admins/registro -H "Content-Type:
application/json" \
  -d '{"nombre":"Admin","email":"admin@correo.com","password":"secreto123"}'

curl http://localhost:8080/Interfaz de programación/restaurantes     # solo activos; incluye hash de
contraseña de cada uno
```

Productos

Operación	Cuerpo	Notas
POST /Interfaz de programación/productos	{"nombre","descripcion","precio","disponible","restaurante":{"id":N}}	restaurante es obligatorio (nullable=false) [Inferido de la entidad]; No se comprueba que exista ni que pertenezca al solicitante
GET /Interfaz de programación/productos/restaurante/{id}	—	Devuelve también productos No disponibles; la app los filtra
PUT /Interfaz de programación/productos/{id}	Producto	Solo actualiza nombre, descripción, precio y disponibilidad
Eliminar /Interfaz de programación/productos/{id}	—	deleteById; puede fallar por clave foránea si el producto está en un pedido

```
curl -X POST http://localhost:8080/Interfaz de programación/productos -H "Content-Type: application/json" \
  -d '{"nombre":"Pizza pepperoni","descripcion":"Mozzarella y
pepperoni","precio":65.0,"disponible":true,"restaurante":{"id":2}}'
curl http://localhost:8080/Interfaz de programación/productos/restaurante/2
curl -X PUT http://localhost:8080/Interfaz de programación/productos/1 -H "Content-Type: application/json" \
  -d '{"nombre":"Pizza pepperoni XL","descripcion":"Grande","precio":80.0,"disponible":true}'
curl -X Eliminar http://localhost:8080/Interfaz de programación/productos/1
```

Pedidos

```
# Crear (versión A: sin ubicación, pago ni envío)
curl -X POST http://localhost:8080/Interfaz de programación/pedidos -H "Content-Type: application/json" \
  -d
'{"clienteId":1,"restauranteId":2,"items":[{"productoId":1,"cantidad":2},{"productoId":3,"cantidad":1}]}'

curl http://localhost:8080/Interfaz de programación/pedidos/10
curl http://localhost:8080/Interfaz de programación/pedidos/pendientes
curl http://localhost:8080/Interfaz de programación/pedidos/cliente/1
curl http://localhost:8080/Interfaz de programación/pedidos/restaurante/2

# Cambiar estado: cuerpo de texto plano; el servidor elimina las comillas
curl -X PUT http://localhost:8080/Interfaz de programación/pedidos/10/estado -H "Content-Type: text/plain" -
d 'ENTREGADO'

# Asignar repartidor (deja el pedido EN_CAMINO)
curl -X PUT http://localhost:8080/Interfaz de programación/pedidos/10/repartidor/3
```

Respuesta de ejemplo (estructura derivada de las entidades; los valores son ilustrativos):

```
{ "id": 10, "estado": "PENDIENTE", "total": 185.0,
  "cliente": { "id": 1, "nombre": "Ana", "email": "ana@correo.com", "password": "<hash BCrypt>",
"direccionFavorita": "Zona 10" },
  "restaurante": { "id": 2, "nombreComercial": "Pollos Don Chepe", "direccion": "Zona 10", "activo": true,
"...": "..." },
  "repartidor": null,
  "detalles": [ { "id": 21, "producto": { "id": 1, "nombre": "…", "precio": 65.0 }, "cantidad": 2,
"subtotal": 130.0 } ] }
```

Calificaciones

```
curl -X POST http://localhost:8080/Interfaz de programación/calificaciones/pedido/10 -H "Content-Type:
application/json" \
  -d '{"estrellas":5,"comentario":"Excelente"}'
```

Errores: pedido inexistente, No ENTREGADO, ya calificado. No se valida el rango de estrellas.

5.9 Diferencias entre versiones y contrato con la app

Aspecto	Backend A (compilado)	App v10 espera	Consecuencia
Crear pedido	Ignora latitudEntrega, longitudEntrega, metodoPago, costoEnvio (campos desconocidos No fallan con la configuración por defecto de Spring Boot) [Inferido]	Los envía	Datos de entrega y pago **se pierden**
Pedido en la respuesta	Sin ubicación, pago, envío ni calificación	Lee latitudEntrega, longitudEntrega, metodoPago, costoEnvio, calificacionEstrellas	La app usa valores por defecto (0, EFECTIVO)
Restaurante en la respuesta	Sin latitud/longitud	Los lee para calcular el envío	Envío fijo de Q15
GET /pedidos/estado/LISTO	No existe	Lista de pedidos disponibles del repartidor	Pantalla vacía o error
GET /pedidos/repartidor/{id}	No existe	“Mis entregas”	Error
Aceptar / rechazar / código / entregar	No existen	Flujo del comercio y de la entrega	Error; la app muestra el mensaje de transición No permitida
Asignar repartidor	Fuerza EN_CAMINO sin validar estado	Espera que el pedido esté LISTO	Comportamiento diferente
Nombres de estado	Libres	PENDIENTE, ACEPTADO, RECHAZADO, PREPARANDO, LISTO, EN_CAMINO, ENTREGADO, CANCELADO	La documentación previa listaba EN_PREPARACION y LISTO_PARA_RECOGER; el código de la app **No** usa esos nombres

Aspecto	Backend A (compilado)	App v10 espera	Consecuencia
Código de entrega	La app lo genera con SecureRandom y lo guarda en memoria, **pero ApiRepository.crearPedido No lo envía**	Lo consulta al servidor (GET .../codigo)	El servidor es la fuente de verdad en B (generación No verificable)

Conclusión. El backend compilado de `clases.rar` es una versión **anterior** al contrato usado por la app v10. Para usar la app v10 completa se necesita el backend B (`Proy1`, junto con el parche si procede). [Inferido: la documentación previa menciona un `parche-backend-pideya.zip` con cambios de pedido y calificaciones; No se recibió]

Capítulo 6. Base de datos

6.1 Motor y origen del esquema

- **Motor:** MySQL según la documentación previa [Reportado]; el bytecode solo muestra JPA genérico, sin referencias a MySQL.
- **No se entregó ningún script SQL ni migración.** El esquema se genera en ejecución con Hibernate (`ddl-auto=Actualizar` según la documentación previa). Lo que sigue se **deriva de las anotaciones JPA** del backend compilado (versión A) y **No equivale a las tablas realmente creadas**: nombres de columna y tipos dependen de la estrategia de nombres y del dialecto. [Inferido donde se indique]
- Las columnas que añade la versión B (ubicación, pago, envío, código, aprobación) se listan aparte como **[Reportado]**.

6.2 Esquema (versión A, verificado en anotaciones)

`usuarios` — herencia `SINGLE_TABLE`

Discriminador: columna `tipo_usuario` con valores `CLIENTE`, `RESTAURANTE`, `REPARTIDOR`, `ADMIN`.

Columna	Tipo Java	Restricciones	Aplica a
id	Long	PK, IDENTITY	todos
nombre	String	NOT NULL	todos
email	String	NOT NULL, UNIQUE	todos
password	String	NOT NULL (hash BCrypt)	todos
tipo_usuario	discriminador	—	todos
direccion_favorita*	String	nulo para otros roles	CLIENTE
nombre_comercial, *direccion*	String	nulos para otros roles	RESTAURANTE
activo*	boolean	por defecto true en el constructor	RESTAURANTE
vehiculo, *disponible*	String, boolean	disponible por defecto true	REPARTIDOR

* Nombre de columna **[Inferido]** (convención camelCase → snake_case de Spring Boot; no hay `@Column(name=...)` explícito). `ADMIN` No añade columnas.

`productos`

Columna	Restricciones
id	PK, IDENTITY
nombre	NOT NULL
descripcion	nulo permitido
precio (Double)	NOT NULL
disponible (boolean)	por defecto true
restaurante_id	NOT NULL, FK → usuarios.id

pedidos

Columna	Restricciones
id	PK, IDENTITY
cliente_id	NOT NULL, FK → usuarios.id
restaurante_id	NOT NULL, FK → usuarios.id
repartidor_id	nulo permitido, FK → usuarios.id
estado (String)	NOT NULL; por defecto PENDIENTE en el constructor
total (Double)	NOT NULL

Relación inversa `detalles`: `@OneToMany(mappedBy="pedido", cascade=ALL, orphanRemoval=true)`.

detalle_pedido

Columna	Restricciones
id	PK, IDENTITY
pedido_id	NOT NULL, FK → pedidos.id (campo @JsonIgnore para evitar recursión)
producto_id	NOT NULL, FK → productos.id
cantidad (Integer)	NOT NULL
subtotal (Double)	NOT NULL

calificaciones

Columna	Restricciones
id	PK, IDENTITY
pedido_id	NOT NULL, **UNIQUE**, FK → pedidos.id (una calificación por pedido)
estrellas (Integer)	NOT NULL (sin CHECK de rango)
comentario	nulo permitido

6.3 Columnas adicionales de la versión B [Reportado]

`pedidos`: latitud y longitud de entrega, método de pago (por defecto `EFECTIVO`), costo de envío, código de entrega y, probablemente, estrellas de calificación expuestas al cliente. `usuarios`: indicador `aprobado` en restaurantes y repartidores. Los nombres exactos No se pueden confirmar sin el código fuente.

6.4 Diagrama entidad–relación

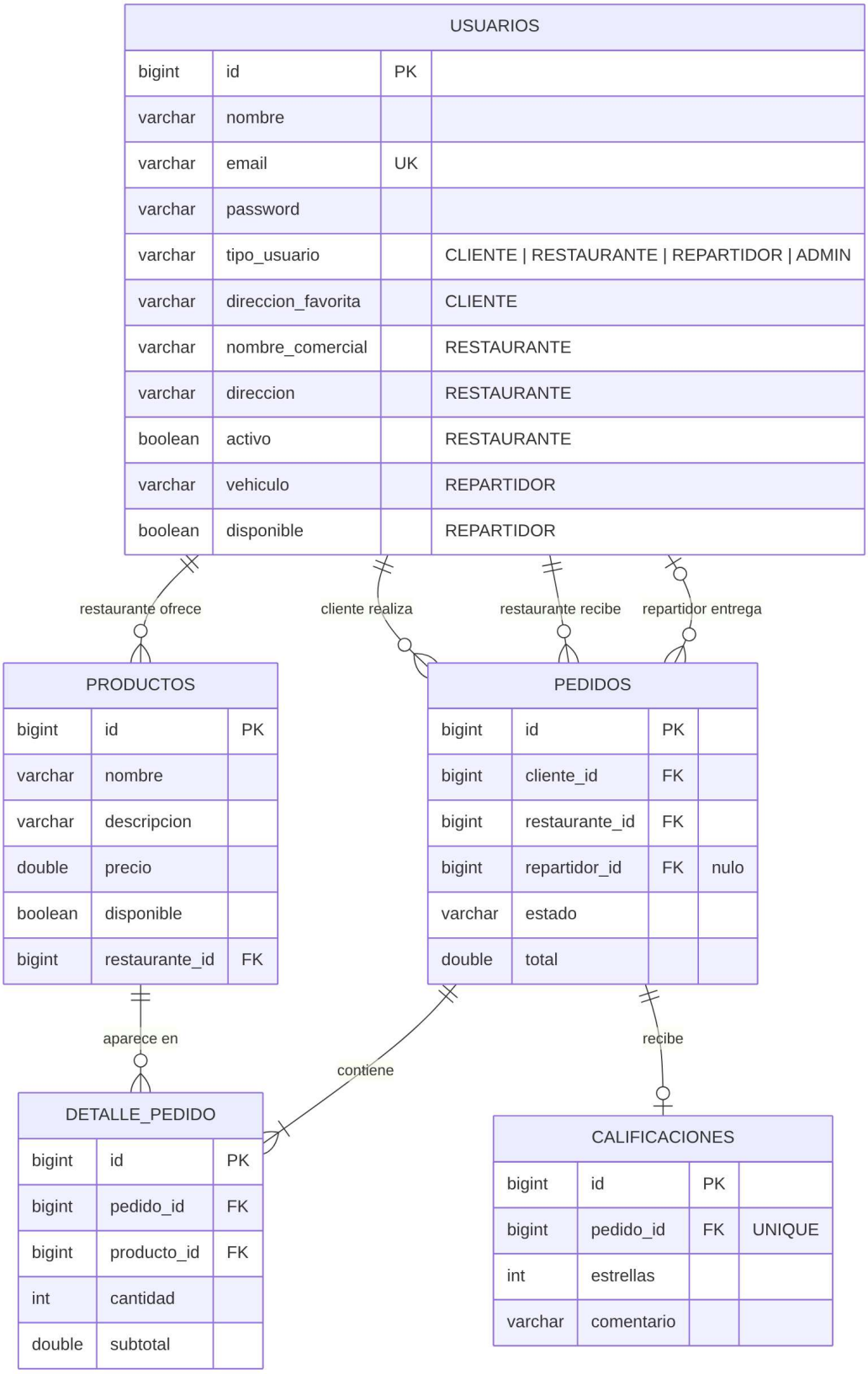


Diagrama 06.1

6.5 Índices y restricciones identificados

- Únicos: `usuarios.email`, `calificaciones.pedido_id`.

- Claves foráneas implícitas por las relaciones `@ManyToOne` / `@OneToOne`.
- No hay `@Index` explícito. Consultas frecuentes sin índice dedicado en MySQL (salvo el que crea InnoDB para claves foráneas): `pedidos.estado`.
- No hay restricciones `CHECK` (cantidad > 0, estrellas 1–5, estados válidos).
- **Los FK de `pedidos` y `productos` apuntan a `usuarios` sin verificar el rol**: la base permitiría que `cliente_id` apunte a un restaurante. El servicio hace el *cast* y fallaría con `ClassCastException` (500).

6.6 DDL orientativo (No generado, No ejecutado)

```
-- Referencia para entender el modelo; Hibernate crea el esquema real.
Crear TABLE usuarios (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  tipo_usuario VARCHAR(31) NOT NULL,
  nombre VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  direccion_favorita VARCHAR(255),
  nombre_comercial VARCHAR(255),
  direccion VARCHAR(255),
  activo BIT(1),
  vehiculo VARCHAR(255),
  disponible BIT(1)
);
Crear TABLE productos (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(255) NOT NULL, descripcion VARCHAR(255),
  precio DOUBLE NOT NULL, disponible BIT(1) NOT NULL,
  restaurante_id BIGINT NOT NULL, FOREIGN KEY (restaurante_id) Referencias usuarios(id)
);
Crear TABLE pedidos (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  cliente_id BIGINT NOT NULL, restaurante_id BIGINT NOT NULL, repartidor_id BIGINT NULL,
  estado VARCHAR(255) NOT NULL, total DOUBLE NOT NULL,
  FOREIGN KEY (cliente_id) Referencias usuarios(id),
  FOREIGN KEY (restaurante_id) Referencias usuarios(id),
  FOREIGN KEY (repartidor_id) Referencias usuarios(id)
);
Crear TABLE detalle_pedido (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  pedido_id BIGINT NOT NULL, producto_id BIGINT NOT NULL,
  cantidad INT NOT NULL, subtotal DOUBLE NOT NULL,
  FOREIGN KEY (pedido_id) Referencias pedidos(id),
  FOREIGN KEY (producto_id) Referencias productos(id)
);
Crear TABLE calificaciones (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  pedido_id BIGINT NOT NULL UNIQUE, estrellas INT NOT NULL, comentario VARCHAR(255),
  FOREIGN KEY (pedido_id) Referencias pedidos(id)
);
```

6.7 Datos de prueba

- **Backend:** No hay semillas (`data.sql`, `import.sql`) en los archivos entregados.
- **App (`MockRepository`)** solo para el modo demostración: 4 restaurantes (Pizza Roma, Pollos Don Chepe, Café Central, Sushi Kai), 8 productos y 3 cuentas demo (`cliente@test.com`, `repartidor@test.com`, `comercio@test.com`; la contraseña está en el código fuente de la clase y es trivial). Estos datos **No existen en la base de datos real**.

6.8 Creación, migraciones, respaldos y restauración

Tema	Situación	Recomendación
Creación	ddl-auto=Actualizar crea/actualiza tablas al arrancar [Reportado]	Usar validate y un script de esquema versionado
Migraciones	No hay Flyway/Liquibase	Adoptar Flyway; partir del DDL generado real

Tema	Situación	Recomendación
Respaldo	No documentado	mysqldump --single-transaction delivery_db > respaldo.sql
Restauración	No documentada	mysql delivery_db < respaldo.sql en una base vacía

6.9 Reglas de integridad de datos

1. Correo único para todos los roles (restricción en BD + comprobación previa).
2. Un pedido siempre tiene cliente, restaurante, estado y total.
3. Los detalles se crean y eliminan junto con el pedido (cascade + orphanRemoval).
4. Una calificación por pedido (restricción única).
5. **No garantizado:** rol correcto en las claves foráneas, estados válidos, cantidades positivas, estrellas entre 1 y 5, coherencia entre `total` y la suma de detalles.
6. Importes en `Double`: se recomienda `DECIMAL`/`BigDecimal`.

Capítulo 7. Aplicaciones Android y web

7.1 Versiones disponibles

Componente	Entregado	Fuente	Estado de análisis
Android **v10** (PideYa-v10-android.zip)	Correcto:	Código fuente Java + recursos XML (65 entradas)	Leído completo, **sin compilar ni ejecutar**
Android v5	Incorrecto:	Mencionada en la documentación previa	Sin analizar; No se pueden afirmar diferencias
Web (index.html, prueba-ubicacion, rayo-1.html, Delivery frontend.zip)	Incorrecto:	Descrita en la documentación previa	[Reportado]

Por eso este capítulo No compara v5 con v10. Solo se puede afirmar lo que la v10 contiene.

7.2 Android v10

Estructura de archivos

```
PideYa/app/src/main/
├── AndroidManifest.xml
├── java/com/pideya/app/                (26 archivos .java)
│   ├── PideYaApp.java                  aplica el tema guardado al iniciar
│   ├── data/    Repo · DeliveryRepository · ApiRepository · MockRepository · Callback
│   ├── model/   Usuario(abstracta) → Cliente · Comercio · Repartidor
│   │            Restaurante · Producto · Pedido · Entrega · EstadoPedido(enum)
│   ├── ui/      LoginActivity · RegistroActivity · RestaurantesActivity · MenuActivity
│   │            PedidosActivity · PedidosComercioActivity · EntregasActivity · ListaAdapter
│   ├── util/    Sesion · Ubicacion · Ui
│   └── {model,data,ui,util}/          ← carpeta vacía con nombre literal (artefacto de un mkdir mal expandido)
└── res/  layout/ (8) · drawable/ (3) · anim/ (6) · values/ y values-night/ (colores y tema)
```

No hay `build.gradle`, `Ajustes.gradle`, `gradle.properties`, íconos (`mipmap`) ni `strings.xml`: **el ZIP No es un proyecto compilable por sí solo**.

Pantallas y navegación

Pantalla	Rol	Función
LoginActivity (launcher)	Todos	Correo y contraseña; redirige a usuario.pantallaInicial()
RegistroActivity	Cliente	Valida campos vacíos, formato de correo y contraseña ≥ 5; registra y entra

Pantalla	Rol	Función
RestaurantesActivity	Cliente	Lista de restaurantes, botón “Mis pedidos”, cambio de tema, cierre de sesión; solicita permiso de ubicación
MenuActivity	Cliente	Menú, carrito en memoria, resumen con envío estimado y método de pago, crea el pedido
PedidosActivity	Cliente	Historial (más recientes primero), código de entrega cuando está EN_CAMINO, calificación
PedidosComercioActivity	Comercio	Pedidos del restaurante; acciones según el estado; ver entrega en Google Maps
EntregasActivity	Repartidor	Alterna “Pedidos disponibles” (estado LISTO) y “Mis entregas”; tomar pedido, confirmar con código, ver mapa

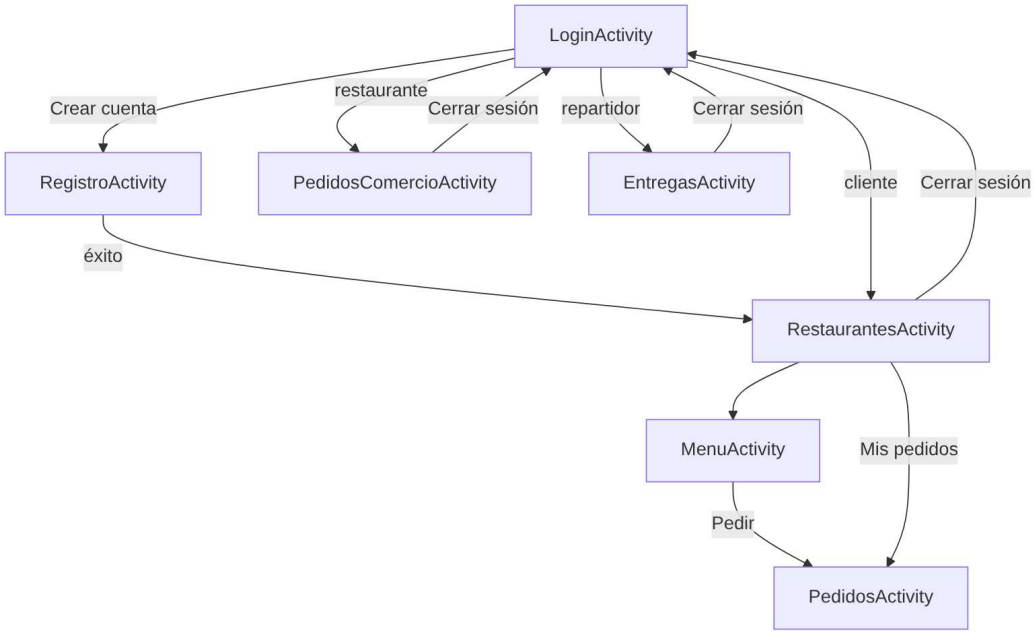


Diagrama 07.1

Comunicación con el backend

`Repo.get()` devuelve un `DeliveryRepository` único; con `USAR_BACKEND = true` es `ApiRepository` (Retrofit + Gson, llamadas asíncronas con `Callback<T>{onOk, onError}`).

Operación de la app	Punto de acceso
login	POST clientes/login
registrar	POST clientes/registro
getRestaurantes	GET restaurantes
getMenu	GET productos/restaurante/{id} (omite disponible=false)
crearPedido	POST pedidos (agrupa el carrito por producto: productId + cantidad)
getPedidos	GET pedidos/cliente/{id}
getPedidosComercio	GET pedidos/restaurante/{id}
getPedidosDisponibles	GET pedidos/estado/LISTO
getMisEntregas	GET pedidos/repartidor/{id}
tomarPedido	PUT pedidos/{id}/repartidor/{rep}
cambiarEstado	PUT .../aceptar, .../rechazar o .../estado (cuerpo text/plain)
entregar	PUT pedidos/{id}/entregar (cuerpo text/plain = código)

Operación de la app	Punto de acceso
getCodigo	GET pedidos/{id}/codigo
calificar	POST calificaciones/pedido/{id}

Conversión: el JSON se transforma a mano (`JsonObject`) en los modelos de la app. **El rol se deduce de los campos presentes**: `nombreComercial` → restaurante, `vehiculo` → repartidor, `direccionFavorita` → cliente; si no hay ninguno, se trata como administrador y se rechaza. Es una dependencia frágil: si el servidor omitiera campos nulos, un cliente sin dirección sería tratado como administrador.

Sesión y tokens

No existen tokens ni persistencia de sesión. `Sesion` guarda en variables estáticas el usuario, el restaurante elegido, el carrito, la ubicación y cachés de calificaciones, pagos y códigos. Al cerrar la app o al llamar `Sesion.cerrar()` todo se pierde; el login se repite en cada arranque. Las solicitudes HTTP **No llevan credenciales**.

URL del servidor

`ApiRepository.BASE_URL = "http://10.0.2.2:8080/Interfaz de programación/"` (constante en el código).

Entorno	Valor
Emulador Android	10.0.2.2 (alias del equipo anfitrión)
Teléfono físico	http://<IP-LAN-del-PC>:8080/Interfaz de programación/ (misma red, firewall abierto al 8080)
Producción	https://... (exige retirar usesCleartextTraffic)

Permisos y manifiesto

Permiso / ajuste	Uso
INTERNET	Llamadas al backend
ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION	Ubicación de entrega (se piden en RestaurantesActivity y MenuActivity)
usesCleartextTraffic="true"	Permite HTTP sin cifrar (solo desarrollo)
PideYaApp como Application	Aplica el tema guardado
Solo LoginActivity está exported	Correcto: el resto No es accesible desde otras apps

Sin permiso de ubicación se usa Ciudad de Guatemala (14.6349, −90.5069) y el diálogo lo indica (“ubicación aproximada”).

Dependencias y compilación

Deducidas de los `import` (sin versiones, ver Cap. 4): AndroidX AppCompat, RecyclerView, Activity, Core, Material Components 3, Retrofit 2 + converter-gson, OkHttp 3, Gson y Play Services Location. Para compilar hacen falta `build.gradle` de proyecto y de módulo, `Ajustes.gradle`, ícono y `strings` (o ajustar el manifiesto).

Funcionalidades verificadas por lectura de código

Correcto: Login/registro con validación local · Correcto: lista de restaurantes y menú · Correcto: carrito con cantidades agrupadas · Correcto: resumen con subtotal, envío y total · Correcto: elección efectivo/tarjeta (solo etiqueta) · Correcto: historial con estados de color · Correcto: calificación 1–5 con comentario · Correcto: flujo del comercio con transiciones válidas · Correcto: flujo del repartidor con código · Correcto: tema claro/oscuro · Correcto: cierre de sesión con confirmación. Ninguna se ejecutó: son **comprobaciones estáticas**.

Observaciones sobre el código (hallazgos)

#	Hallazgo	Impacto
1	LAT_ENTREGA/LNG_ENTREGA en ApiRepository están declaradas y **No se usan**; las coordenadas reales vienen de Ubicacion.actual() (GPS o ubicación por defecto). La documentación previa decía que v10 “fija” coordenadas provisionales: es inexacto	Documentación corregida
2	El código de entrega se genera en MenuActivity (SecureRandom, 4 dígitos) pero **crearPedido No lo envía**; con Interfaz de programación solo sirve el código del servidor (getCodigo)	El cliente genera un valor que se descarta
3	El costo de envío se calcula en el cliente y se envía; el servidor No debería confiar en él	Riesgo de manipulación
4	El método de pago y el envío se guardan además en Sesion.pagos (solo en memoria)	Se pierden al reiniciar
5	Existe la carpeta literal {model,data,ui,util} vacía	Limpieza
6	El manifiesto declara las actividades con sangría inconsistente y sin ícono	Cosmético / compilación
7	Sin indicadores de carga ni reintentos; las listas se refrescan al reanudar o tocando “actualizar”	Usabilidad
8	PedidosComercioActivity usa el ID del usuario como ID de restaurante (correcto solo porque ambos comparten la tabla usuarios)	Acoplamiento
9	En MockRepository el comercio demo y el cliente demo comparten el ID 1	Solo afecta al modo demostración

7.3 Aplicación web [Reportado]

La documentación previa describe páginas HTML autónomas con `fetch` hacia `/Interfaz de programación/clientes/login` y `/registro`, dos páginas de prueba de ubicación y `rayo-1.html` (función No confirmada). **No se recibió ningún archivo web**, por lo que No se documentan estructura, pantallas ni dependencias. Se afirma únicamente que, de existir, dependería de CORS abierto (`*`) para llamar a la Interfaz de programación desde otro origen.

7.4 Diferencias entre versiones

- **Android v10 vs backend A:** incompatibles en 6 rutas y varios campos (ver §5.9).
- **Android v5 vs v10:** No verificable (v5 No entregada).
- **Backend A vs B:** B añade aprobación de cuentas, disponibilidad de repartidores, aceptar/rechazar, código, ubicación, pago y envío [Reportado].

Capítulo 8. Flujos, reglas y algoritmos

8.1 Ciclo de vida del pedido

Las reglas de transición **solo existen en la app** (`EstadoPedido.puedePasarA`). El backend compilado acepta cualquier texto como estado.

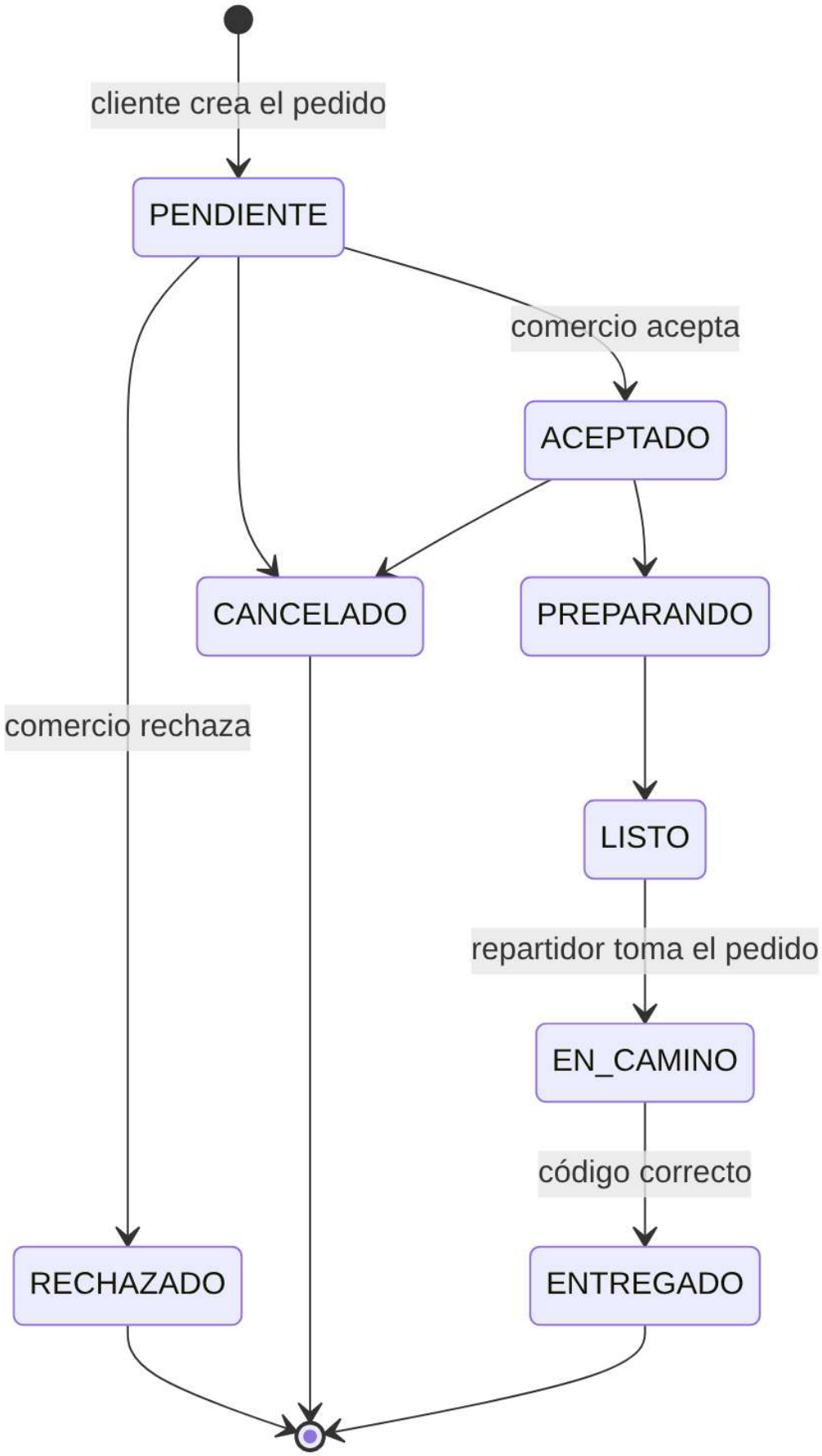

cliente crea el pedido
PENDIENTE
comercio acepta
ACEPTADO
CANCELADO
PREPARANDO
comercio rechaza
LISTO
repartidor toma el pedido
EN_CAMINO
código correcto
RECHAZADO
ENTREGADO

Diagrama 08.1

Estado	Etiqueta en la app	Quién lo provoca
PENDIENTE	En espera: Pendiente	Creación (valor por defecto del servidor)
ACEPTADO / RECHAZADO	Correcto: / Incorrecto:	Comercio
PREPARANDO → LISTO	/ Paquete:	Comercio
EN_CAMINO		Repartidor (asignación)
ENTREGADO		Repartidor con código
CANCELADO		Previsto en las reglas; ninguna pantalla v10 lo dispara

> **Corrección a la documentación previa:** esta usaba `EN_PREPARACION` y `LISTO_PARA_RECOGER`. El código de la app usa `PREPARANDO` y `LISTO`.

8.2 Creación de un pedido

Entrada: `clienteId`, `restauranteId`, `items[{productoId, cantidad}]` (en B además ubicación, pago y envío). **Salida:** `Pedido` con `detalles` y `total`.

```
crearPedido(req):
  cliente      ← buscar usuario por req.clienteId       (error "Cliente no encontrado")
  restaurante  ← buscar usuario por req.restauranteId   (error "Restaurante no encontrado")
  pedido       ← nuevo Pedido(cliente, restaurante)     # estado = PENDIENTE
  total ← 0
  para cada item en req.items:
      producto ← buscar producto por item.productoId    (error "Producto no encontrado")
      detalle  ← nuevo DetallePedido(pedido, producto, item.cantidad)
      detalle.subtotal ← producto.precio × item.cantidad
      pedido.detalles.añadir(detalle); total ← total + detalle.subtotal
  pedido.total ← total
  guardar(pedido)                                       # los detalles se guardan en cascada
```

Ejemplo. 2 × Pizza (Q65) + 1 × Pasta (Q48) → subtotales Q130 y Q48 → **total Q178**. El total del servidor **No incluye el envío** (en B se guarda aparte).

No valida: `items` vacío, cantidad ≤ 0, producto no disponible, producto de otro restaurante, ni que cliente/restaurante tengan el rol correcto (el *cast* fallaría con 500).

8.3 Asignación de repartidor

- **A:** `PUT /pedidos/{id}/repartidor/{rep}` busca pedido y usuario (cast a `Repartidor`), asigna y fija **`EN_CAMINO`** sin comprobar el estado previo ni la disponibilidad.
- **App:** solo muestra "Tomar pedido" cuando el estado es `LISTO`; en B el servidor responde error si No lo está ("Debe estar LISTO").
- **Asignación automática por cercanía:** No existe; faltan coordenadas del repartidor.

8.4 Confirmación por código de 4 dígitos

1. La app genera un código con `SecureRandom` (`%04d`) en `MenuActivity`; con `ApiRepository` **No se envía**.
2. Mientras el pedido está `EN_CAMINO`, el cliente toca el pedido y la app pide `GET …/codigo` al servidor.
3. El repartidor pide el código al cliente y llama `PUT …/entregar` con el texto.
4. Si es correcto → `ENTREGADO`; si No, la app muestra "Código incorrecto, o el pedido No está en camino".

Con solo 10 000 combinaciones y sin límite de intentos documentado, el código es vulnerable a fuerza bruta (ver Cap. 9). La generación y validación en el servidor (B) No son verificables.

8.5 Cálculo del costo de envío (en la app)

$$\text{envío} = \min(60,\ 8 + 2.5 \times d_{km})$$

con d_{km} calculada con la fórmula de Haversine (radio 6371 km) entre el restaurante y la ubicación del cliente; **Q15 fijos** si el restaurante No tiene coordenadas; redondeo a 2 decimales.

Restaurante (demo)	Distancia a la ubicación por defecto	Envío
Pizza Roma (14.6200, −90.5150)	1.87 km	**Q12.68**
Sushi Kai (14.6000, −90.4850)	4.54 km	**Q19.35**
Restaurante real con backend A (sin coordenadas)	—	**Q15.00**

Cálculos reproducidos con la misma fórmula del código; No se ejecutó la app.

8.6 Calificación

`calificar(pedidoId, estrellas, comentario)`: error si el pedido No existe, si su estado No es `ENTREGADO` o si ya hay una calificación. La app envía `max(1, round(rating))`; el servidor **No valida el rango**.

8.7 Inicio de sesión y deducción del rol

```
login(email, password):
  usuario ← UsuarioRepository.findByEmail(email)      # error "Usuario no encontrado"
  si !BCrypt.matches(password, usuario.password): error "Contraseña incorrecta"
  devolver usuario                                    # incluye el hash en el JSON
App: rol ← nombreComercial ? restaurante : vehiculo ? repartidor : direccionFavorita ? cliente : admin
     si admin → error "Esta app es para clientes, repartidores y comercios."
```

Capítulo 9. Seguridad y calidad

Revisión **estática** (bytecode y código Android). No es una auditoría: No se ejecutó nada ni se analizaron dependencias vulnerables (No hay `pom.xml`/Gradle).

Clasificación: confirmado (visto en el código) · riesgo potencial · no verificable con los archivos entregados.

9.1 Hallazgos

ID	Sev.	Estado	Hallazgo	Evidencia	Recomendación
S-01	Crítica		**Ninguna ruta exige autenticación.** Cualquiera puede leer pedidos, cambiar estados, borrar productos o crear administradores	SecurityConfig: anyRequest().permitAll(); CSRF desactivado	Autenticación (sesión segura o JWT) y reglas hasRole por ruta
S-02	Crítica		**Registro de administradores abierto**: POST /Interfaz de programación/admins/registro sin restricción	AdminController	Crear admins solo desde otro admin o por script
S-03	Alta		**Se devuelve el hash de contraseña** en login, registro, GET /Interfaz de programación/restaurantes (público) y dentro de cada Pedido	Usuario.password sin @JsonIgnore; solo DetallePedido lo usa	DTO de respuesta o @JsonIgnore/@JsonProperty(access = WRITE_ONLY)
S-04	Alta		**IDOR**: cualquier ID de cliente/restaurante/pedido es accesible y modificable	Rutas /cliente/{id}, /{pedidoId}/estado, etc.	Comprobar que el recurso pertenece al usuario autenticado
S-05	Alta		Estado del pedido sin validar; asignarRepartidor fuerza EN_CAMINO	PedidoService	Máquina de estados en el servidor
S-06	Media		CORS abierto: allowedOrigins("*") en /api/**	CorsConfig$1	Lista blanca de orígenes
S-07	Media		Tráfico sin cifrar en Android	usesCleartextTraffic="true", http://	HTTPS; retirar el flag fuera de desarrollo
S-08	Media		Errores como RuntimeException → 500; el mensaje de login distingue “usuario No encontrado” de “contraseña incorrecta” (enumeración de cuentas)	Servicios	Excepciones tipadas, 401/404/409 y mensaje único en login
S-09	Media		Sin validación de entradas (cantidad ≤ 0, estrellas fuera de rango, items vacío)	Servicios y DTO sin anotaciones	Bean Validation en DTO y servicios

ID	Sev.	Estado	Hallazgo	Evidencia	Recomendación
S-10	Media		Código de entrega de 4 dígitos sin límite de intentos documentado	Diseño de la app; servidor No verificable	Límite de intentos, caducidad, aumentar longitud
S-11	Media		El costo de envío lo calcula y envía el cliente	Ui.costoEnvio, crearPedido	Calcular en el servidor
S-12	Media		Credencial de BD en texto plano en application.properties	Documentación previa; archivo No entregado	Variables de entorno; **rotar la contraseña si fue compartida**
S-13	Baja		Importes en Double	Entidades	BigDecimal / DECIMAL(10,2)
S-14	Baja		Sesión solo en memoria; solicitudes sin credenciales	Sesion.java	Token con expiración almacenado de forma segura
S-15	Baja		Cuentas demo con contraseña trivial en MockRepository	Código fuente	No publicar con USAR_BACKEND=false
S-16	Info		ddl-auto=Actualizar, show-sql=true, dependencias vulnerables, límites de tasa, registro de eventos	No verificable	Revisar con los archivos de configuración reales

Lo que sí está bien: contraseñas con **BCrypt** (registro y comparación con `matches`) para los cuatro roles; permisos de Android mínimos y solo `LoginActivity` exportada; el correo se normaliza por recorte en la app.

9.2 Privacidad

La ubicación del cliente se obtiene con consentimiento del sistema (permiso en tiempo de ejecución) y se envía al servidor (B). Conviene informar el uso, minimizar la precisión (ya se usa `PRIORITY_BALANCED_POWER_ACCURACY`) y No registrar coordenadas en logs.

9.3 Calidad y mantenibilidad

Aspecto	Observación
Pruebas	Solo contextLoads() según B; ninguna en la app
Separación en capas	Buena; servicios acotados
Inyección	Por campo (@Autowired); preferible por constructor
Duplicación	Los cuatro registrar* repiten la misma lógica
Validación	Ausente en el servidor; solo en el cliente (fácil de saltar)
Documentación en código	Comentarios útiles en la app; ninguno comprobable en el backend (compilado)
Versionado	Varias copias del backend y de la app sin una versión canónica

9.4 Prioridades antes de publicar

1. Rotar y externalizar credenciales. 2. Autenticación y autorización por rol y por propietario. 3. DTO de respuesta sin `password`. 4. HTTPS y CORS restringido. 5. Máquina de estados y validaciones en el servidor. 6. Cálculo de envío y total en el servidor. 7. Pruebas de autorización y de flujo de pedido. 8. Una sola versión canónica de backend y app.

9.5 Matriz de permisos objetivo (propuesta, No es el comportamiento actual)

Hoy **todos los roles pueden hacerlo todo** (S-01). La documentación previa propone esta matriz; se conserva como especificación para implementar con autenticación, roles y comprobación de propiedad del recurso.

Operación	Cliente	Restaurante	Repartidor	Admin
Consultar restaurantes y productos	Sí	Sí	Sí	Sí
Crear pedido	Sí	No	No	Según necesidad

Operación	Cliente	Restaurante	Repartidor	Admin
Consultar pedidos	Propios	Solo de su comercio	Solo asignados	Sí
Aceptar/rechazar pedidos	No	Solo propios	No	Supervisión
Asignar repartidor	No	Según reglas	No (en la app v10 el repartidor “toma” pedidos LISTO)	Sí
Confirmar entrega	No	No	Solo asignados	Supervisión
Aprobar cuentas	No	No	No	Sí
Cambiar disponibilidad del repartidor	No	No	Propia	Sí
Calificar pedido entregado	Solo el cliente propietario	No	No	Moderación
Crear/editar/borrar productos	No	Solo propios	No	Sí
Registrar administradores	No	No	No	Sí

Rol explícito: devolver el rol en la respuesta de login en lugar de que la app lo deduzca por campos opcionales del JSON.

Capítulo 10. Pruebas y validación

10.1 Qué se hizo en este análisis

Actividad	Resultado
Extracción de clases.zip → clases.rar (RAR5)	Correcto: 32 .class extraídos
Lectura del bytecode con javap (firmas, anotaciones, constantes, instrucciones)	Correcto:
Lectura completa del código fuente Android v10 y de sus recursos	Correcto:
Contraste con el documento consolidado (DOCX)	Correcto:
Cálculo de ejemplos (Haversine y envío)	Correcto: con Python, misma fórmula
Descompilación a .java	Incorrecto: No se pudo descargar un descompilador (red restringida)
Compilación, ejecución, pruebas, conexión a BD, app en emulador	Incorrecto: **No realizadas**

10.2 Pruebas existentes

- Backend (B): `DeliveryBackendApplicationTests.contextLoads()` con `@SpringBootTest`; No está en el paquete compilado. [Reportado]
- App: No hay carpeta `test` ni `androidTest`. [Verificado]
- La clase `MockRepository` ofrece datos para probar manualmente la interfaz sin servidor.

10.3 Plan de pruebas recomendado

Área	Casos	Tipo
Registro/login	Correo duplicado, contraseña errónea, campos vacíos, hash en BD, ausencia de password en la respuesta	Interfaz de programación
Productos	CRUD, precio negativo, producto de otro restaurante, borrar producto con pedidos	Interfaz de programación

Área	Casos	Tipo
Pedidos	Carrito vacío, cantidad 0/negativa, producto No disponible, IDs inexistentes, total = Σ	Interfaz de programación + unitaria (PedidoService)
Estados	Transición válida/ inválida, doble entrega, pedido rechazado	Interfaz de programación
Asignación	Repartidor No disponible, pedido No LISTO, repartidor inexistente	Interfaz de programación
Calificación	0, 6 estrellas; pedido No entregado; segunda calificación	Interfaz de programación
Seguridad	Rutas sin sesión, acceso cruzado entre usuarios, cliente llamando a rutas de admin	Interfaz de programación
Android	Servidor caído, JSON incompleto, rol desconocido, permiso de ubicación denegado, rotación de pantalla	Instrumentadas / manual
Contrato app↔backend	Cada ruta de ApiRepository.Interfaz de programación contra el backend elegido	Contrato

10.4 Verificación de rutas sin ejecutar

Para confirmar el contrato sin levantar el sistema basta comparar `ApiRepository.Interfaz de programación` con los controladores (tabla de §5.7). Resultado: **16 rutas usadas por la app; 10 existen en A; las 6 restantes (más campos de datos) solo en B**.

10.5 Comandos sugeridos

```
./mvnw test            # backend (Windows: mvnw.cmd test)
./mvnw clean package
./gradlew assembleDebug testDebugUnitTest    # app (requiere los archivos Gradle)
```

Regla: No declarar éxito hasta observar código de salida 0 y los reportes de pruebas.

Capítulo 11. Instalación y configuración

Procedimiento **previsto**; No se ejecutó. Con los archivos entregados **No se puede construir el backend** (solo hay `.class`) **ni la app** (faltan los archivos Gradle).

11.1 Requisitos previos

Componente	Requisito
Backend	JDK 17+, MySQL en ejecución, puerto 8080 libre, acceso a Maven Central
App	Android Studio, SDK acorde al proyecto Gradle, emulador o teléfono
Red	Equipo y teléfono en la misma red para pruebas físicas

11.2 Opción 1 — backend a partir del código fuente (`Proy1`)

```
# 1) Base de datos (usuario con privilegios limitados)
mysql -u root -p -e "Crear Base de datos delivery_db CHARACTER SET utf8mb4;
  Crear Usuario 'pideya'@'localhost' IDENTIFIED BY '<contraseña-nueva>';
  GRANT ALL ON delivery_db.* TO 'pideya'@'localhost';"

# 2) Variables de entorno (Linux/macOS)
export DB_URL="jdbc:mysql://localhost:3306/delivery_db"
export DB_USER=pideya DB_PASSWORD='<contraseña-nueva>'

# 3) Ejecutar
cd delivery-backend && chmod +x mvnw && ./mvnw spring-boot:run
# Windows: mvnw.cmd spring-boot:run

# 4) Comprobar
curl -i http://localhost:8080/Interfaz de programación/restaurantes     # lista JSON (posiblemente vacía)
```

Ajusta `application.properties` para leer `DB_*` (ver §5.5). No reutilices la contraseña del archivo original.

11.3 Opción 2 — ejecutar las clases compiladas de `clases.rar` (solo verificación local)

Requiere reconstruir un JAR ejecutable con las dependencias de Spring Boot (No incluidas); sin `pom.xml` es más sencillo recuperar el proyecto `Proy1`. [Pendiente]

11.4 App Android

1. Abrir la carpeta `PideYa` en Android Studio y **agregar los archivos Gradle** del proyecto original (con las dependencias de §4.2).
2. Ajustar `ApiRepository.BASE_URL` (emulador `10.0.2.2`; teléfono: IP LAN).
3. Para una demostración sin servidor: `Repo.USAR_BACKEND = false` (cuentas demo en `MockRepository`).
4. Ejecutar y conceder permiso de ubicación (o usar la ubicación por defecto).

11.5 Crear datos mínimos para probar (backend en marcha)

```
curl -X POST localhost:8080/Interfaz de programación/restaurantes/registro -H "Content-Type:
application/json" \
  -d '{"nombre":"Luis","email":"r@x.com","password":"clave12345","nombreComercial":"Pizza
Roma","direccion":"Zona 4"}'
curl -X POST localhost:8080/Interfaz de programación/productos -H "Content-Type: application/json" \
  -d '{"nombre":"Pizza","descripcion":"Pepperoni","precio":65,"disponible":true,"restaurante":{"id":1}}'
curl -X POST localhost:8080/Interfaz de programación/clientes/registro -H "Content-Type: application/json" \
  -d '{"nombre":"Ana","email":"c@x.com","password":"clave12345","direccionFavorita":"Zona 10"}'
```

(los IDs dependen del orden de creación: la tabla `usuarios` es única).

Capítulo 12. Despliegue y mantenimiento

12.1 Preparación para un entorno compartido

- Elegir **un repositorio canónico** y etiquetar versiones (hoy conviven backend compilado A, backend fuente B, Android v5/v10 y copias `Proy1`).
- Configuración por entorno con variables: URL/usuario/contraseña de BD, URL pública de la Interfaz de programación, orígenes CORS. Nada de secretos en el repositorio.
- HTTPS (proxy inverso o certificado en el servicio) y perfiles `dev`/`prod`.
- Desactivar `show-sql`; sustituir `ddl-auto=Actualizar` por migraciones (Flyway) y `validate`.
- Registro de eventos sin datos personales, monitoreo, copias de seguridad programadas y prueba de restauración.

12.2 Construcción

```
./mvnw clean package      # JAR en target/; reconstruir siempre desde la versión elegida
java -jar target/<artefacto>.jar --spring.profiles.active=prod
```

Los JAR ya presentes en las copias entregadas No demuestran corresponder al código actual (según la documentación previa).

12.3 App Android

Cambiar `BASE_URL` por la URL HTTPS, quitar `usesCleartextTraffic`, fijar `USAR_BACKEND = true`, firmar el APK/AAB y probar en dispositivo físico. Acordar con el backend los nombres de campos y estados (ver §5.9).

12.4 Cambios de esquema

1. Revisar relaciones y restricciones. 2. Escribir la migración. 3. Probarla con una copia de la BD. 4. Verificar compatibilidad con la versión anterior del backend. 5. Documentar el cambio y su reversión.

12.5 Mantenimiento de la documentación

Actualizar la tabla de Puntos de acceso cuando cambien los controladores y los diagramas cuando cambien las entidades. Registrar en cada revisión a qué versión de backend y de app corresponde.

12.6 Backlog técnico sugerido

Prioridad	Tarea
P0	Autenticación/autorización, DTO sin password, rotación de credenciales
P0	Unificar backend (B) con la app v10 y verificar el contrato
P1	Máquina de estados y validaciones en el servidor; manejo de errores con códigos HTTP correctos
P1	Envío y total calculados en el servidor; BigDecimal
P2	Flyway, pruebas automatizadas, CI
P2	Asignación por cercanía (coordenadas del repartidor), panel de administración

Capítulo 13. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
El backend No arranca	JDK < 17, puerto 8080 ocupado, BD inaccesible	java -version; liberar el puerto; leer el primer error de la consola
Error de conexión MySQL	Servicio apagado, usuario sin permisos, driver No descargado	Verificar el servicio y los permisos; configurar credenciales por entorno
La app muestra “No se pudo conectar con el servidor”	Backend apagado, URL incorrecta, firewall	Emulador 10.0.2.2; teléfono con IP LAN; abrir el puerto 8080
Todo funciona en la app pero con datos ficticios	USAR_BACKEND=false	Poner true
“Esta app es para clientes, repartidores y comercios.”	Cuenta de administrador, o JSON sin los campos de rol	Usar otra cuenta; revisar que el servidor incluya direccionFavorita/nombreComercial/vehiculo
“Correo o contraseña incorrectos.”	El servidor devolvió 500 por usuario inexistente o clave errónea (la app No distingue)	Revisar el log del backend
“No se pudo cambiar el estado…” / “Código incorrecto…”	Transición No permitida, o la ruta No existe en el backend usado (A)	Usar el backend B
Pantalla de repartidor vacía o con error	El backend No tiene /pedidos/estado/{estado} ni /pedidos/repartidor/{id}	Usar el backend B
“No se pudo calificar…”	Pedido No ENTREGADO o ya calificado	Normal
Envío siempre Q15	El restaurante No devuelve latitud/longitud	Añadir coordenadas al restaurante en el backend
La app No compila	Faltan archivos Gradle, íconos, strings	Agregar los archivos del proyecto original
Respuestas JSON con campo password	Entidades expuestas sin DTO	Ver S-03

Capítulo 14. Glosario

Término	Definición
Interfaz de programación REST	Interfaz HTTP que expone recursos en JSON
BCrypt	Algoritmo de hash de contraseñas con sal
CORS	Mecanismo del navegador que decide qué orígenes pueden llamar a una Interfaz de programación

Término	Definición
CSRF	Ataque de solicitud falsificada; aquí está desactivado
DTO	Objeto de transferencia de datos (LoginRequest, PedidoRequest…)
Entidad JPA	Clase mapeada a una tabla
Haversine	Fórmula de distancia entre dos coordenadas
IDOR	Acceso a recursos ajenos cambiando un identificador
JPA / Hibernate	Especificación y motor de mapeo objeto-relacional
Mock	Implementación simulada con datos en memoria
Polimorfismo	Varias subclases implementan el mismo método (obtenerPanel, pantallaInicial)
Retrofit	Cliente HTTP declarativo para Android
SINGLE_TABLE	Estrategia JPA: toda la jerarquía en una tabla con discriminador
Discriminador	Columna (tipo_usuario) que indica el rol de cada fila
Q	Quetzal, moneda de Guatemala
Versión A / B / C	Backend compilado / backend fuente (reportado) / app Android v10

Capítulo 15. Fuentes y trazabilidad

Fuente	Contenido	Uso	Fiabilidad
clases.zip → clases.rar	32 .class del backend (**A**)	Controladores, servicios, entidades, repositorios, config	Alta (bytecode); sin comentarios ni nombres de parámetros
PideYa-v10-android.zip (**C**)	Código fuente Java y recursos de la app	Capítulo 7 y contrato con la Interfaz de programación	Alta; sin Gradle
Documentacion_Completa_PideYa.docx (**B**)	Documento consolidado previo	Backend fuente Proy1, web, versiones, configuración, requisitos	Media: No verificable aquí; contiene errores corregidos en el Cap. 16
Documentacion_PideYa.zip	24 archivos .md previos	Estructura y límites del análisis anterior	Media
Prompts maestros (.txt)	Instrucciones de documentación	Estructura de capítulos	—

No recibidos: `rar.zip`, `Proy1 .zip`, `Proy1 - 01.zip`, `Delivery frontend.zip`, `parche-backend-pideya.zip`, `PideYa-v5-android.zip`, HTML sueltos, PDF “estado y roles”, `PROYECTO PROGRA.txt`.

Convención: cada afirmación está etiquetada **[Verificado]**, **[Reportado]**, **[Inferido]** o **[Pendiente]** (ver §1.9).

Prioridad aplicada: cuando el DOCX y el código entregado se contradicen, **prevalece el código** (bytecode de A y código de la app).

Capítulo 16. Auditoría de la documentación previa

Se contrastó el documento consolidado (`Documentacion_Completa_PideYa.docx`, 20 capítulos + diagramas e inventario) y los 24 `.md` anteriores con el código **realmente entregado**: el bytecode del backend compilado (A) y la app Android v10 (C). Donde el código prevalece sobre la documentación, se indica.

16.1 Correcciones confirmadas por el código

ID	Documentación previa dice	El código muestra	Acción

ID	Documentación previa dice	El código muestra	Acción
C-01	v10 “usa coordenadas de entrega provisionales” / “No debe interpretarse como GPS” (§18.4, §7.5)	Ubicacion.actual() usa la ubicación real (LocationManager) y Ubicacion.actualizar() el *Fused Location*; solo cae a Ciudad de Guatemala sin permiso. LAT_ENTREGA/LNG_ENTREGA están declaradas y **sin uso**	Corregido en §7.2
C-02	Estados EN_PREPARACION y LISTO_PARA_RECOGER (§10.2)	EstadoPedido: PENDIENTE, ACEPTADO, RECHAZADO, PREPARANDO, LISTO, EN_CAMINO, ENTREGADO, CANCELADO	Diagrama rehecho (§8.1)
C-03	“La estrategia de herencia y los nombres de tabla deben confirmarse” (§9.1)	SINGLE_TABLE, discriminador tipo_usuario (CLIENTE/RESTAURANTE/REPARTIDOR/ADMIN), tablas usuarios, productos, pedidos, detalle_pedido, calificaciones	Resuelto en el Cap. 6
C-04	El ER trata CLIENTE, RESTAURANTE, REPARTIDOR, ADMIN como entidades separadas	Son filas de **una sola tabla**	ER reemplazado
C-05	“Confirmar que las contraseñas se codifican con BCrypt” (§13.2)	Confirmado para los 4 roles en A (encode al registrar, matches al entrar)	RNF-01 Correcto:
C-06	“Devolver entidades **puede** incluir password” (§13.2)	**Sí lo incluye**, también en GET /Interfaz de programación/restaurantes público y en cada Pedido	Hallazgo S-03
C-07	“No se debe inferir que el listado filtra por aprobación” (§8.2)	En A filtra solo por activo; No existe aprobado	RN-03
C-08	El código de entrega “debe auditarse” (§10.4)	La app lo genera con SecureRandom pero **No lo envía** en crearPedido; consulta el del servidor	§8.4
C-09	Rutas de la app “coinciden en gran medida con el backend inspeccionado” (§18.3)	Cierto para B; **falso para el backend compilado A**: faltan 6 rutas	§5.9
C-10	clases.rar “No se pudo verificar” (§17, Limitaciones)	Se extrajo: 32 .class (backend A)	Documentado
C-11	Ambigüedad posible entre /pedidos/{pedidoId} y /pedidos/pendientes	Existen ambas en A; Spring prioriza el patrón literal más específico; riesgo bajo, pero conviene una prueba	Nota en pruebas
C-12	“El ZIP Android v5 es más pequeño…” y diferencias v5/v10	v5 No entregada	Sin afirmaciones sobre v5

16.2 Contradicciones y vacíos que persisten

ID	Tema	Situación
V-01	Dos backends (A compilado, B fuente Proy1)	A es anterior al contrato de la app v10; falta saber si el parche parche-backend-pideya.zip cierra la brecha
V-02	RutaController.java	Aparece en Delivery frontend.zip pero No en la lista de controladores de B ni en A: posible función de rutas/mapas No documentada
V-03	Spring Boot 4.1.1 y dependencias	Solo en la documentación previa (pom.xml No entregado)
V-04	Código de entrega en el servidor	Generación, longitud y límite de intentos No verificables
V-05	Versiones web y Android v5	Sin archivos
V-06	Estados que acepta el servidor B	No verificable; la app asume los 8 listados
V-07	Cálculo de envío: ¿valida o recalcula el servidor B?	No verificable

16.3 Defectos de forma del DOCX consolidado

1. **Enlaces rotos:** aparecen como texto literal (`Introducción`) en el índice y las notas, porque apuntan a archivos `.md` que No existen dentro del DOCX.
2. **Diagramas Mermaid como texto plano** (citas de bloque), No como imágenes.
3. **Numeración de listas continua entre secciones** (los pasos de §7.1 empiezan en 11; §10.1 en 18; §13.3 en 22; §14.4 en 32; "Limitaciones" en 37).
4. **El cuerpo repite el README y el índice** (con enlaces a archivos externos) antes del capítulo 1, y los diagramas e inventario quedan como secciones sin numerar al final.
5. **Ejemplos y tablas con marcas de código incrustadas** (acentos graves literales en las tablas).
6. Mezcla de dos propósitos (documento de proyecto y registro de análisis) en el mismo archivo.

16.4 Mejoras aplicadas en esta versión

- Estructura por capítulos del encargo (1–7) más seguridad, pruebas, instalación, despliegue, problemas, glosario, fuentes y esta auditoría.
- Cada afirmación lleva etiqueta de certeza y la fuente (A/B/C).
- Tabla de Puntos de acceso con **18 rutas verificadas + 9 reportadas** y su uso en la app.
- Esquema de BD, DDL orientativo y ER fiel a las anotaciones; diagramas de contexto, contenedores, componentes, secuencia, estados y flujo de pantallas.
- Matriz de trazabilidad requisito → módulo → evidencia.
- Ejemplos `curl` para los 18 Puntos de acceso verificados.
- Enlaces internos funcionales y diagramas renderizados como imágenes en el DOCX.

16.5 Mejoras prioritarias pendientes (para cerrar el análisis)

1. **Entregar** `Proy1 .zip` (código fuente B, `pom.xml`, `application.properties`, Pruebas) y el parche, para sustituir lo **[Reportado]** por **[Verificado]** y documentar `CuentaController`.
2. Entregar los archivos **Gradle** de la app y, si existe, la v5.
3. Entregar la **web** (`index.html`, pruebas de ubicación, `rayo-1.html`, `RutaController`).
4. Entregar `PROYECTO PROGRA.txt` y el PDF de estado y roles para completar los requisitos y el estado oficial.
5. Compilar, levantar MySQL y probar la lista de pruebas del Cap. 10.

Anexo — Inventario de archivos analizados

Archivos recibidos

Archivo	Contenido	Revisión
ad76231b-attachment.txt (×2 prompts)	Instrucciones de análisis y de mejora (ambos cortados al final)	Leídos
Documentacion_PideYa.zip	24 .md de documentación previa	Leídos (README, índice, inventario) y contrastados
Documentacion_Completa_PideYa.docx	Documento consolidado (20 capítulos + anexos)	Leído completo
clases.zip → clases.rar	32 archivos .class del backend (**A**)	**Analizado** con javap
PideYa-v10-android.zip	65 entradas: 26 .java, manifiesto, 8 layouts, 3 drawables, 6 animaciones, 2 archivos de valores (**C**)	**Leído completo**

Backend compilado (A) — 32 clases

Paquete	Clases	Estado
raíz	DeliveryBackendApplication	Analizada
config	SecurityConfig, CorsConfig, CorsConfig$1	Analizadas
controller	Usuario, Restaurante, Repartidor, Admin, Producto, Pedido, Calificacion (+Controller)	Analizadas (rutas, parámetros, anotaciones)
model	Usuario, Cliente, Restaurante, Repartidor, Admin, Producto, Pedido, DetallePedido, Calificacion, LoginRequest, PedidoRequest (+$ItemPedido), CalificacionRequest	Analizadas (campos, JPA)
repository	4 interfaces	Analizadas
service	UsuarioService, ProductoService, PedidoService, CalificacionService	Analizadas (bytecode instrucción a instrucción en los métodos de negocio)

Parcialmente revisado: cuerpos de `equals/hashCode/toString` (generados, sin interés) y las lambdas internas de `listarRestaurantes` (se confirmó el filtro por `Restaurante` y `isActivo`).

Android v10 (C)

Grupo	Archivos	Estado
data	Repo, DeliveryRepository, ApiRepository, MockRepository, Callback	Leídos (Mock: cabecera y lógica relevante; resto de datos de demostración revisado por muestreo)
model	9 clases	Leídas
ui	7 actividades + ListaAdapter	Leídas
util	Sesion, Ubicacion, Ui	Leídas (Ui: funciones relevantes)
res	Layouts, drawables, animaciones, temas	Inventariados; se verificó la definición de los colores usados; los XML de layout No se leyeron en detalle
Manifiesto	AndroidManifest.xml	Leído

No recibidos

`rar.zip`, `Proy1 .zip`, `Proy1 - 01.zip`, `Delivery frontend.zip`, `parche-backend-pideya.zip`, `PideYa-v5-android.zip`, HTML sueltos, PDF de estado y roles, `PROYECTO PROGRA.txt`, archivos Gradle, `pom.xml`, `application.properties`.

Hallazgos de higiene

- Carpeta vacía literal `app/src/main/java/com/pideya/app/{model,data,ui,util}/`.
- `ApiRepository.LAT_ENTREGA/LNG_ENTREGA` sin uso.
- Entrada de `clases.zip` que contiene un RAR (No ZIP), formato RAR5.

Anexo — Limitaciones del análisis

1. **El backend se analizó desde bytecode**, sin fuentes: No hay comentarios, nombres de parámetros ni Lombok visible; las conclusiones sobre lógica proceden de las instrucciones (`javap -c`). No fue posible descargar un descompilador (red restringida).
2. **No se entregaron** `pom.xml`, `application.properties`, SQL ni el código fuente `Proy1`: versiones de librerías, conexión MySQL y `CuentaController` son **[Reportados]** por la documentación previa.
3. **La app No incluye archivos Gradle**: No se pueden determinar `minSdk`, versiones ni compilar.
4. **Web y Android v5 No se recibieron**; No se documentan.
5. **No se compiló ni ejecutó nada**: ni backend, ni app, ni pruebas, ni base de datos.
6. Nombres de columna y tipos SQL son **derivados** de las anotaciones y de la convención de Spring Boot; pueden diferir de las tablas reales.
7. El comportamiento HTTP 500 ante `RuntimeException` y la omisión de campos desconocidos al deserializar son los **valores por defecto de Spring Boot**; No se probaron.
8. Los ejemplos `curl` y las respuestas JSON se construyeron a partir de las entidades y No se ejecutaron.
9. No se hizo análisis de dependencias vulnerables, ni pruebas dinámicas de seguridad.
10. La relación entre backend A y B (¿A es una versión anterior?) es una **inferencia** apoyada en que la app v10 usa rutas que A No tiene y en que el documento previo describe esas rutas en el backend fuente.

Para cerrarlas: entregar `Proy1 .zip` (con `pom.xml`, `application.properties`, Pruebas), el parche, los archivos Gradle y la web; luego compilar, levantar MySQL y ejecutar el plan de pruebas.